The Most Efficient Protocol for PAKE:

What Exact Stuff Do You Need to Hash at the End?

Jiayu Xu

Oregon State University, xujiay@oregonstate.edu

1 Introduction

2 Preliminaries and Technical Overview

2.1 KEM Properties Used in the "Hash Everything" Version

Definition 1. *A KEM scheme is* ***anonymous with*** q ***uniform public keys and random oracles (***q***-UNI-ANO-RO)*** *if the following two distributions are indistinguishable:*

$pk_1, \ldots, pk_q \leftarrow \mathbb{G}$	$pk', pk_1, \ldots, pk_q \leftarrow \mathbb{G}$
$i \leftarrow \mathcal{A}(pk_1, \ldots, pk_q)$	$i \leftarrow \mathcal{A}(pk_1, \ldots, pk_q)$
$(c, K) \leftarrow \mathsf{Encap}(pk_i)$	$(c, \star) \leftarrow \mathsf{Encap}(pk')$
$h_1 := H_1(K); h_2 := H_2(K)$	$h_1 \leftarrow \{0,1\}^n; h_2 \leftarrow \{0,1\}^{2n}$
$b \leftarrow \mathcal{A}(c, h_1, h_2)$	$b \leftarrow \mathcal{A}(c, h_1, h_2)$

Here, H_1 *and* H_2 *are ROs with range* $\{0,1\}^n$ *and* $\{0,1\}^{2n}$*, respectively.*

This property emerges when the OEKE adversary $\mathcal{A}$ sends (s^*, T^*) to P' which contains a wrong password guess. In the real world, (s^*, T^*) decrypt to a uniformly random KEM public key, which is then used to compute c, SK, τ (the left game). In simulation, we want to remove the decryption process and outright use a freshly sampled public key to compute c, SK, τ (the right game). (h_1, h_2 in Definition 1 correspond to SK, τ in the protocol, respectively.) What complicates things is that $\mathcal{A}$ can decrypt a large number of (s^*, T^*) pairs *under both correct and incorrect passwords*, resulting in a large number of pk^* values, *before* sending (s^*, T^*) to P' (or even before initiating P''s session); at this time the reduction does not know which (s^*, T^*) will be used by $\mathcal{A}$, or which password is correct, yet it has to program the public key (the "right" one that will be used in P''s algorithm) into the decryption process. As such, we need to give the reduction polynomially many public keys.

2.2 Additional KEM Properties Used in Other Versions

Definition 2. *A KEM scheme is* ***weakly non-malleable (WNM)*** *if for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins the following game is negligible:*

$(pk, sk) \leftarrow \mathsf{Gen}$
$(c, K) \leftarrow \mathsf{Encap}(pk)$
$c^* \leftarrow \mathcal{A}(pk, c)$
abort if $c^* = c$
$\mathcal{A}$ wins if $\mathsf{Decap}(sk, c^*) = K$

3 Security Analysis of the "Hash Everything" Version

The main theorem in this section is:

Theorem 1. *Suppose the UNI-PK, OW-PCA and ANO-PCA properties hold for the KEM scheme. Then the OEKE protocol as described in* **??** *UC-realizes $\mathcal{F}_{\text{PAKE-1EA}}$, in the random oracle model for H_1, H_2, H_{key} and H_{tag}.*

The rest of this section is dedicated to the proof of Theorem 1. In Section 3.1 we describe the simulator, and in Section 3.2 we argue why the simulator generates a view indistinguishable from the real-world view.

3.1 The Simulator

3.1.1 Simulation Strategy

A large portion of the simulator is routine; below we explain the idea behind the non-trivial parts.

Simulating honest P-to-P' message. When P's session begins, $\mathcal{S}$ must simulate its message (s, T) to P'. This can be easily done: just sample (s, T) at random. However, every time $\mathcal{A}$ queries $H_2(\mathsf{pw}^*, T)$ (let the result be t^*), $\mathcal{S}$ needs to generate a KEM key pair (pk^*, sk^*) for later use (see the "Extracting password guess from malicious P'" paragraph below) and make sure that (pk^*, sk^*) is consistent with the RO queries. This can be done as follows: $\mathcal{S}$ computes

$$r^* := s \oplus t^*, \quad R^* := T/pk^*,$$

and programs $H_1(\mathsf{pw}^*, r^*) := R^*$. This also allows $\mathcal{S}$ to store the association between pw^* and (pk^*, sk^*). (Note that (pk^*, sk^*) needs to be freshly generated every time $\mathcal{A}$ queries $H_2(\star, T)$; otherwise R^* never changes and $\mathcal{A}$ can easily find collisions in H_1.)

Extracting password guess from malicious P. When the adversary $\mathcal{A}$ sends a message (s^*, T^*) to P', if it is different from the honest message (s, T), the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding $\mathsf{TestPwd}$ command to $\mathcal{F}_{\text{PAKE-1EA}}$. There are two types of attack, which need to be addressed separately:

1. *"Normal" online attack:* $\mathcal{A}$ executes P's algorithm on a password guess pw^*. That is, $\mathcal{A}$ samples public key pk^* and randomness r^*, computes

$$R^* := H_1(\mathsf{pw}^*, r^*), \qquad T^* := pk^* \cdot R^*,$$
$$t^* := H_2(\mathsf{pw}^*, T^*), \qquad s^* := r^* \oplus t^*,$$

   and sends (s^*, T^*) to P'.
   It takes a while to realize how to extract pw^* from (s^*, T^*). First, $\mathcal{S}$ should scan all $H_2(\star, T^*)$ queries, which yields multiple pw^* values (and multiple t^* values); $\mathcal{S}$ must decide which one is the "actual" password guess. This can be done via the following. For every $H_2(\mathsf{pw}^*, T^*)$ query (let the result be t^*), $\mathcal{S}$ computes the corresponding $r^* := s^* \oplus t^*$, and checks if (and when) $H_1(\mathsf{pw}^*, r^*)$ was queried. *With overwhelming probability, there is at most one such H_1 query made before the $H_2(\mathsf{pw}^*, T^*)$ query*, from which $\mathcal{S}$ can extract the "actual" password guess pw^*.

2. *Related key attack:* This attack works only after $\mathcal{A}$ receive an honest (s, T) pair from P. $\mathcal{A}$ skips sampling pk^* or r^*; instead, $\mathcal{A}$ picks any $\Delta \in \mathbb{G}$, computes $T^* := T \cdot \Delta$ and

$$t^* := H_2(\mathsf{pw}^*, T), \qquad (t^*)' := H_2(\mathsf{pw}^*, T^*),$$
$$\delta := t^* \oplus (t')^*, \qquad s^* := s \oplus \delta,$$

   and sends (s^*, T^*) to P'.
   Let us first explain what $\mathcal{A}$ is trying to achieve here. The same randomness r yields two valid messages, (s, T) and (s^*, T^*), such that

$$s \oplus s^* = H_2(\mathsf{pw}, T) \oplus H_2(\mathsf{pw}, T^*),$$

   which allows $\mathcal{A}$ to pick T^* and "reverse engineer" the corresponding s^* if it guesses pw correctly. (s^*, T^*) implicitly encrypts $pk^* = T^*/H_1(\mathsf{pw}, r)$, which $\mathcal{A}$ can compute as $pk \cdot (T^*/T)$.
   To extract pw^* from (s^*, T^*), $\mathcal{S}$ should scan all $H_2(\star, T)$ and $H_2(\star, T^*)$ queries (which yield multiple candidate pw^* values) and check which ones satisfy

$$H_2(\mathsf{pw}^*, T) \oplus H_2(\mathsf{pw}^*, T^*) = s \oplus s^*.$$

   With overwhelming probability, at most one pw^* will satisfy this equation.

Remark: On the term "anchor query". The term "anchor query" was coined in [MRR20, Section 3.3], where it refers to the special $H_1(\mathsf{pw}^*, r^*)$ query made before the $H_2(\mathsf{pw}^*, T^*)$ query in case (1) above; case (2) is completely

missing in [MRR20]. This error is fixed in [JRX25], which uses "anchor query" to refer to something else: the $H_2(\mathsf{pw}^*, T^*)$ query in either case (1) or (2) (see [JRX25, Section 4.3.1] and [JRX25, Figure 11, step 4]). This term is not used in [ABJ25].

Extracting password guess from malicious P'. When the adversary $\mathcal{A}$ sends a message (c^*, τ^*) to P, if $\mathcal{A}$ is not passive (i.e., it is not the case that $(s^*, T^*) = (s, T)$ *and* $(c^*, \tau^*) = (c, \tau)$), the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding $\mathsf{TestPwd}$ command to $\mathcal{F}_{\text{PAKE-1EA}}$.

$\mathcal{S}$ should find the H_{tag} query whose result is τ^*, which contains $(\mathsf{pw}^*, pk^*, K^*)$. However, pw^* constitutes a password guess only if it is consistent with pk^* and K^*, i.e., $K^* = \mathsf{Decap}(sk^*, c^*)$ for sk^* corresponding to pk^*. $\mathcal{S}$ can check this condition because it has stored the correspondence between pw^* and (pk^*, sk^*) (see the "Simulating honest P-to-P' message" paragraph above), and can use sk^* to check if the equation holds. (This extraction strategy does not work anymore if *either* pw^* *or* pk^* is not included in H_{tag}. In later sections we will explain how to simulate other versions of the protocol.)

3.1.2 Formal Description of the Simulator

Below we use $\mathcal{F}$ as a shorthand for $\mathcal{F}_{\text{PAKE-1EA}}$. Please see Figures 1 and 2 for the simulator $\mathcal{S}$.

P-to-P' message

On $(\mathsf{NewSession}, sid, P, P')$ from $\mathcal{F}$, sample $s \leftarrow \{0,1\}^{3n}, T \leftarrow \mathbb{G}$ and send (s, T) from P to P'.

P''s session key and P'-to-P message

On $(\mathsf{NewSession}, sid, P', P)$ from $\mathcal{F}$ and (s^*, T^*) from $\mathcal{A}$ to P', simulate P''s output and the P'-to-P message as follows:

1. If $(s^*, T^*) = (s, T)$, send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$; generate $(pk', \star) \leftarrow \mathsf{Gen}$, compute $(c, \star) \leftarrow \mathsf{Encap}(pk')$, sample $\tau' \leftarrow \{0,1\}^{2n}$, and send (c', τ') from P' to P.
2. Otherwise find $\mathcal{A}$'s $H_2((\mathsf{pw}')^*, T^*)$ query (let the result be t^*) that satisfies *either* of the following:
   - *Type (1):* $\mathcal{A}$ queried $H_1((\mathsf{pw}')^*, r^*)$ *before* the $H_2((\mathsf{pw}')^*, T^*)$ query, where $r^* = s^* \oplus t^*$.
   - *Type (2):* $\mathcal{A}$ also queried $H_2((\mathsf{pw}')^*, T)$, and
   $$s \oplus s^* = H_2((\mathsf{pw}')^*, T) \oplus t^*.$$

   (a) If there is more than one such $(\mathsf{pw}')^*$, abort.

   (b) If there is exactly one such $(\mathsf{pw}')^*$, send $(\mathsf{TestPwd}, sid, P', (\mathsf{pw}')^*)$ to $\mathcal{F}$; if there is no such $(\mathsf{pw}')^*$, send $(\mathsf{TestPwd}, sid, P', \perp)$ to $\mathcal{F}$.

      i. If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$; sample $pk' \leftarrow \mathbb{G}$, compute $(c, \star) \leftarrow \mathsf{Encap}(pk')$, sample $\tau' \leftarrow \{0,1\}^{2n}$, and send (c, τ') from P' to P.

      ii. If $\mathcal{F}$ replies with "correct guess", and $(\mathsf{pw}')^*$ is defined in type (1) above, then r^* is already defined. Compute
      $$R' := H_1((\mathsf{pw}')^*, r^*), \quad pk' := T^*/R'.$$
      If $\mathcal{F}$ replies with "correct guess", and $(\mathsf{pw}')^*$ is defined in type (2) above, then $\mathcal{A}$ has queried $H_2((\mathsf{pw}')^*, T)$, and a record $\langle (\mathsf{pw}')^*, (pk')^*, \star \rangle$ must have been stored at that time (see step 4 below). Compute
      $$pk' := (pk')^* \cdot (T^*/T).$$
      Either way, compute $(c, K') \leftarrow \mathsf{Encap}(pk')$, $SK' := H_{\mathsf{key}}(K')$, $\tau' := H_{\mathsf{tag}}((\mathsf{pw}')^*, pk', c, K')$, and send (c, τ') from P' to P.

Figure 1: UC simulator $\mathcal{S}$ for OEKE (part 1). Red text denotes difference from the simulator in [ABJ25]

P's session key

On (c^*, τ^*) from $\mathcal{F}$, simulate P's output as follows:

1. If $(s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$. // $\mathcal{A}$ is passive
2. Otherwise find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, c^*, K^*)$ query resulting in τ^*.
   (a) If there is no such query or more than one such query, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
   (b) If there is exactly one such query, retrieve record $\langle \mathsf{pw}^*, \star, sk^* \rangle$. If there is no such record, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
   (c) Compute $K := \mathsf{Decap}(sk^*, c^*)$. If $K^* \neq K$, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
   (d) Send $(\mathsf{TestPwd}, sid, P', \mathsf{pw}^*)$ to $\mathcal{F}$.
      i. If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
      ii. If $\mathcal{F}$ replies with "correct guess", compute $SK := H_{\mathsf{key}}(K)$ and send $(\mathsf{NewKey}, sid, P', SK)$ to $\mathcal{F}$.

RO queries

Answer all of $\mathcal{A}$'s RO queries via lazy sampling, except for the following:
When T is sampled (see step 1), if $\mathcal{A}$ has already queried $H_2(\star, T)$, abort.
On $\mathcal{A}$'s $H_2(\mathsf{pw}^*, T)$ query (let the result be t^*), generate $(pk^*, sk^*) \leftarrow \mathsf{Gen}$ and compute

$$r^* := s \oplus t^*, \quad R^* := T/pk^*.$$

If $H_1(\mathsf{pw}^*, r^*)$ has already been defined and it is not R^*, abort. Otherwise program $H_1(\mathsf{pw}^*, r^*) := R^*$, and store $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$.

Figure 2: UC simulator $\mathcal{S}$ for OEKE (part 2)

Our simulator is, by and large, the same as the one in [ABJ25, Fig.5 and 6], although we completely rewrite it to improve readability. For those who would like to go over both our simulator and the one in [ABJ25], we include a comparison in Section A.

3.2 Game-Hop Argument

We now prove that for any PPT environment $\mathcal{Z}$, the simulator $\mathcal{S}$ in Section 3.1.2 generates an ideal-world view indistinguishable from $\mathcal{Z}$'s real-world view. The sequence of games start from the real world (Game 0) and ends in the ideal world (Game 17).

A game-by-game comparison with the proof in [ABJ25, Section 5] is shown in *red*, with the parts that critically rely on certain items being included as inputs to H_{tag} *underlined*.

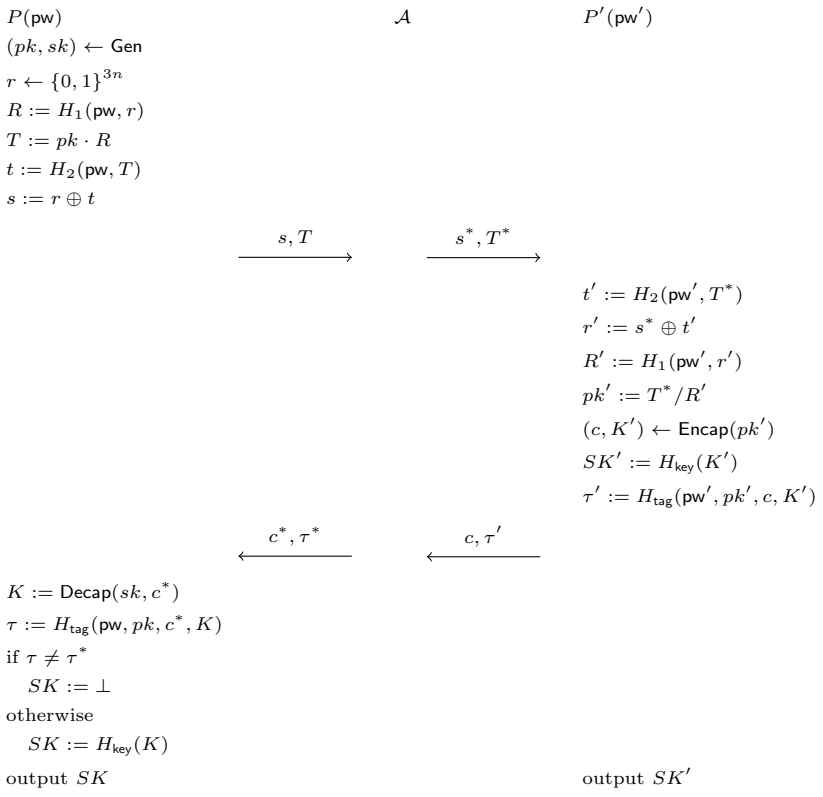

Figure 3: Game 0 (the real world)

Game 0: This is the real world. Recall that the passwords of P and P' are pw and pw', respectively; the flow of events is:

- P sends (s, T) to P';
- It is intercepted by the man-in-the-middle adversary $\mathcal{A}$, who sends (s^*, T^*) (which may or may not be equal to (s, T)) to P';
- P' outputs session key SK' and sends (c, τ') to P;
- It is again intercepted by $\mathcal{A}$, who sends (c^*, τ^*) to P;
- P computes τ. If $\tau^* = \tau$ then P outputs SK, otherwise P outputs $\perp$.

(Note that there are three τ values: τ' computed and sent by P', τ^* sent by $\mathcal{A}$, and τ computed by P used in P's check.) An illustration is shown in Figure 3.

Passwords match and $\mathcal{A}$ is passive, except maybe modifying the tag.

Game 1: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, do the following:

- If $\tau^* = \tau'$ (i.e., $\mathcal{A}$ is passive), set $SK := SK'$.
- If $\tau^* \neq \tau'$, set $SK := \perp$.

By correctness of the protocol (see Section 2), in Game 0 $\tau = \tau'$ except with probability $\mathbf{Adv}^{\text{correctness}}$. This means that except with probability $\mathbf{Adv}^{\text{correctness}}$, P's check $\tau^* \stackrel{?}{=} \tau$ in Game 0 is equivalent to P's check $\tau^* \stackrel{?}{=} \tau'$ in Game 1. If the check passes then both games set $SK := H_{\mathsf{key}}(K)$, otherwise both games set $SK := \perp$. We have

$$\mathbf{Dist}^{0,1}_{\mathcal{Z}} = \mathbf{Adv}^{\text{correctness}}.$$

This game corresponds to games G_3–G_5 in [ABJ25]. We believe having a single game that handles both the $\tau^* = \tau'$ case and the $\tau^* \neq \tau'$ case is much cleaner.

P and P''s passwords match, and $\mathcal{A}$ forwards the P-to-P' message.

Game 2: In the case where $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query resulting in τ^*. If there is none or more than one, set $SK := \perp$. If there is a unique such query, do the following:

- If $K^* = K$, set $SK := H_{\mathsf{key}}(K)$ (i.e., there is no change from Game 1).
- If $K^* \neq K$, set $SK := \perp$.

Claim 1. *The probability that P' queries $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ is at most $(q_1 + 1)/|\mathbb{G}|$.*

P' makes a single H_{tag} query, on $(\mathsf{pw}', pk', c, K')$. If $\mathsf{pw} \neq \mathsf{pw}' \vee c^* \neq c$ then of course P''s query is not $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$. Otherwise $\mathsf{pw} = \mathsf{pw}' \wedge c^* = c$, so $(s^*, T^*) \neq (s, T)$. We upper-bound $\Pr[pk' = pk]$ in this case:

We have

$$pk = \frac{T}{R} = \frac{T}{H_1(\mathsf{pw}, r)} = \frac{T}{H_1(\mathsf{pw}, s \oplus t)} = \frac{T}{H_1(\mathsf{pw}, s \oplus H_2(\mathsf{pw}, T))},$$
$$pk' = \frac{T^*}{R'} = \frac{T^*}{H_1(\mathsf{pw}, r')} = \frac{T^*}{H_1(\mathsf{pw}, s^* \oplus t')} = \frac{T^*}{H_1(\mathsf{pw}, s^* \oplus H_2(\mathsf{pw}, T^*))}.$$

There are two cases:

- $s \oplus H_2(\mathsf{pw}, T) = s^* \oplus H_2(\mathsf{pw}, T^*)$. In this case $T^* \neq T$ (otherwise $T^* = T \Rightarrow H_2(\mathsf{pw}, T^*) = H_2(\mathsf{pw}, T) \Rightarrow s^* = s$, so $(s^*, T^*) = (s, T)$, which is impossible). But then it is impossible that $pk = pk'$.
- $s \oplus H_2(\mathsf{pw}, T) \neq s^* \oplus H_2(\mathsf{pw}, T^*)$. In this case $H_1(\mathsf{pw}, s^* \oplus H_2(\mathsf{pw}, T^*))$ is uniformly random in $\mathbb{G}$ and independent of both pk and T^*, so the probability that $pk = pk'$, i.e., $H_1(\mathsf{pw}, s^* \oplus H_2(\mathsf{pw}, T^*))$ happens to be equal to T^*/pk, is $1/|\mathbb{G}|$ for a single H_1 query.

Since there are $q_1 + 1$ H_1 queries (q_1 by $\mathcal{A}$ and one by P'), the probability that $pk = pk'$ is upper-bounded by $(q_1 + 1)/|\mathbb{G}|$. If $pk \neq pk'$ then of course P''s query is not $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$. This yields the claim.

Going back to the game hop, Game 1 and Game 2 are identical unless of the following happens:

1. $\mathcal{A}$ never makes an $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ query resulting in τ^*, yet P's check $\tau^* \stackrel{?}{=} H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ passes.
2. $\mathcal{A}$ makes two different $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ queries, both resulting in τ^*.
3. $\mathcal{A}$ makes an $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query resulting in τ^*, and P's $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ query also results in τ^*.

In the first event, by Claim 1 P' does not query $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ except with probability up to $(q_1+1)/|\mathbb{G}|$. $\mathcal{A}$ itself also does not query $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ (otherwise the result is τ^*). So $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ is independent of $\mathcal{A}$'s view, and the probability that $\mathcal{A}$ sends $\tau^* = H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ is $1/2^{2n}$. The second and third events are collision in H_{tag}, whose (combined) probability is upper-bounded by $q_{\mathsf{tag}}(q_{\mathsf{tag}} + 1)/2^{2n+1}$. Overall,

$$\mathbf{Dist}^{1,2}_{\mathcal{Z}} \leq \frac{q_{\mathsf{tag}}(q_{\mathsf{tag}} + 1) + 2}{2^{2n+1}} + \frac{q_1 + 1}{|\mathbb{G}|}.$$

This game corresponds to games G_6 and G_7 in [ABJ25]. However, there are two differences:

1. *The protocol in [ABJ25] additionally includes (s, T) as part of P's input to H_{tag}. Then Claim 1 is trivial (and the probability in the claim is 0): P's H_{tag} query is on $(\mathsf{pw}, pk, s, T, c^*, K)$ and P''s H_{tag} query is on $(\mathsf{pw}', pk', s^*, T^*, c, K')$; if they are equal then $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, violating the assumption of this game.*
2. *The H_{tag} collision cases (events (2) and (3) above) were ruled out in game G_2 in [ABJ25] (as aborting condition [A1]).*

The proof of Claim 1 critically relies on pw, pk, c being included in H_{tag} inputs: pw and c have to be included in order to rule out the $\mathsf{pw} \neq \mathsf{pw}' \vee c^* \neq c$ case; pk has to be included because in the $\mathsf{pw} = \mathsf{pw}' \wedge c^* = c$ case, we show that P and P''s H_{tag} queries are different by comparing P's query input pk and P''s query input pk'.

Game 3: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, sample $SK', \tau' \leftarrow \{0,1\}^n$. ($c$ is still computed as before.)

Let Query_1 be the event that $\mathcal{A}$ queries *either* $H_{\mathsf{key}}(\mathsf{pw}, pk, c, K')$ *or* $H_{\mathsf{tag}}(\mathsf{pw}, pk, c, K')$. Clearly Game 2 and Game 3 are identical unless Query_1 happens. We upper-bound $\Pr[\mathsf{Query}_1]$ via a reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property of KEM:

Reduction $\mathcal{R}_{\text{OW-1PCA1}}$: // Boxed text indicates the single PC oracle query; same for reductions below

Sample $i \leftarrow [q_{\mathsf{key}} + q_{\mathsf{tag}}]$ as a guess that $\mathcal{A}$'s i-th H_{key} or H_{tag} query triggers Query_1.
On input (pk, c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T), except that $\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk rather than $(pk, \star) \leftarrow \mathsf{Gen}$. (Note that computing (s, T) does not require knowing sk.) Send (s, T) to $\mathcal{A}$.

2. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{OW-1PCA1}}$'s inputs.)

3. On (c^*, τ^*) from $\mathcal{A}$,

   (a) If $c^* = c$, do what's described in Game 1.

   (b) If $c^* \neq c$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)

   (c) $\boxed{\text{Query } \mathsf{PC}(sk, c^*, K^*)}$. // This is a valid query since we do this step only if $c^* \neq c$
   If the result is 1, output $SK := H_{\mathsf{key}}(K^*)$ to $\mathcal{Z}$.
   If the result is 0, output $SK := \perp$ to $\mathcal{Z}$.

4. On $\mathcal{A}$'s i-th H_{key} or H_{tag} query, if it is in the form of $H_{\mathsf{key}}((K')^*)$ or $H_{\mathsf{tag}}(\mathsf{pw}, pk, c, (K')^*)$ for some $(K')^*$, output $(K')^*$; otherwise abort.

Assuming Query_1 happens, the only difference between Game 2/3 and $\mathcal{R}_{\text{OW-1PCA1}}$'s simulation (until Query_1 happens) is that $\mathcal{R}_{\text{OW-1PCA1}}$ uses its input pk on the P side, and then uses its input $c \leftarrow \mathsf{Encap}(pk)$ on the P' side. Since we assume $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, in Game 2/3 P''s algorithm will recover $pk' = pk$, so there c is also computed as $\mathsf{Encap}(pk)$ and thus $\mathcal{R}_{\text{OW-1PCA1}}$ simulates Game 2/3 perfectly. Furthermore, if $\mathcal{R}_{\text{OW-1PCA1}}$'s guess is correct then it wins the OW-1PCA game. We have

$$\mathbf{Dist}^{2,3}_{\mathcal{Z}} \leq \Pr[\mathsf{Query}_1] \leq (q_{\mathsf{key}} + q_{\mathsf{tag}})\mathbf{Adv}^{\text{OW-1PCA}}_{\mathcal{R}_{\text{OW-1PCA1}}}.$$

This game corresponds to game G_8 in [ABJ25]. However, there are two differences:

1. *[ABJ25] reduces to OW-PCA (where the KEM adversary can query its plaintext-checking oracle polynomially many times), rather than OW-1PCA. In this case, step 4 of the reduction above can query $\mathsf{PC}(sk, c, (K')^*)$ $q_{\mathsf{key}} + q_{\mathsf{tag}}$ times to check which of $\mathcal{A}$'s query (if any) triggers Query_1, hence avoiding the $q_{\mathsf{key}} + q_{\mathsf{tag}}$ loss in its advantage.*

2. *The above also means that the plaintext-checking oracle in their OW-PCA game has to allow for* $\mathsf{PC}(sk, c, \star)$ *queries, whereas our reduction to OW-1PCA never makes a query in this form and thus the plaintext-checking oracle in our OW-1PCA game can disallow such queries.*

Game 4: Again in the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, generate P''s KEM public key $(pk', \star) \leftarrow \mathsf{Gen}$ and compute $(c, \star) \leftarrow \mathsf{Encap}(pk')$. (Note that K' is not used anywhere since Game 3 — in particular, P' does not use K' to compute SK' or τ' anymore — so we do not need to explicitly mention it.)

The difference between Game 3 and Game 4 is that in Game 3 both P and P' use the same pk, whereas in Game 4 P uses pk and P' uses an independent pk'. We construct a reduction $\mathcal{R}_{\text{ANO-1PCA}}$ to the ANO-1PCA property of KEM. This reduction, in fact, is identical to the reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property in Game 3 except the last step; for completeness, we still present the full reduction and highlight the difference in blue:

Reduction $\mathcal{R}_{\text{ANO-1PCA}}$:

On input (pk, c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T), except that $\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk rather than $(pk, \star) \leftarrow \mathsf{Gen}$. (Note that computing (s, T) does not require knowing sk.) Send (s, T) to $\mathcal{A}$.

2. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{ANO-PCA1}}$'s inputs.)

3. On (c^*, τ^*) from $\mathcal{A}$,

   (a) If $c^* = c$, do what's described in Game 1.

   (b) If $c^* \neq c$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)

   (c) $\boxed{\text{Query } \mathsf{PC}(sk, c^*, K^*)}$. // This is a valid query since we do this step only if $c^* \neq c$
   If the result is 1, output $SK := H_{\mathsf{key}}(K^*)$ to $\mathcal{Z}$.
   If the result is 0, output $SK := \perp$ to $\mathcal{Z}$.

4. When $\mathcal{Z}$ outputs bit b^*, copy $\mathcal{Z}$'s output.

$\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk on the P side and c on the P' side. If $b = 0$ then $c \leftarrow \mathsf{Encap}(pk)$, i.e., both P and P' use the same pk, so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 3; if $b = 1$ then $c \leftarrow \mathsf{Encap}(pk')$, i.e., P' uses an independently generated pk', so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 4. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{3,4} \leq \mathbf{Adv}^{\text{ANO-1PCA}}_{\mathcal{R}_{\text{ANO-1PCA}}}.$$

This game corresponds to game G_9 in [ABJ25]. However, there are two differences:

1. *[ABJ25] reduces to ANO-PCA (where the KEM adversary can query its plaintext-checking oracle polynomially many times), rather than ANO-1PCA. Unlike in Game 3, here reducing to this stronger assumption does not provide any advantage.*
2. *The assumption [ABJ25] reduces to is even further stronger in that their KEM adversary's inputs are (pk, pk', c), whereas we do not give pk' to the KEM adversary. Note that the reduction does not need pk' at all (it only needs to send c which is an encapsulation of either pk or pk'), so the assumption in [ABJ25] is unnecessarily strong.*

P and P''s passwords do not match, and $\mathcal{A}$ forwards the P-to-P' message.

Game 5: After P sends (s, T) aimed at P', when $\mathcal{A}$ makes a fresh query $H_2(\mathsf{pw}^*, T)$ for $\mathsf{pw}^* \neq \mathsf{pw}$ (let the result be t^*), generate $(pk^*, sk^*) \leftarrow \mathsf{Gen}$, and compute

$$r^* := s \oplus t^*, \quad R^* := T/pk^*.$$

If $H_1(\mathsf{pw}^*, r^*)$ has already been defined and it is not R^*, abort. Otherwise program $H_1(\mathsf{pw}^*, r^*) := R^*$, and store $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$.

Furthermore, in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, when P' queries $H_2(\mathsf{pw}', T)$, use the method described above to generate pk', r', R' and program H_1. (Note that the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ has already been addressed in Game 3 and Game 4, and P' does not need to make an H_2 query there.)[1]

In the analysis below, we combine $\mathcal{A}$'s $H_2(\mathsf{pw}^*, T)$ queries and P''s (possible) $H_2(\mathsf{pw}', T)$ query, and call all of them "$H_2(\mathsf{pw}^*, T)$". We first upper-bound the aborting probability. For every fresh $H_2(\mathsf{pw}^*, T)$ query, an independent $t^* \leftarrow \{0,1\}^{3n}$ value is sampled, which results in an independent $r^* \leftarrow \{0,1\}^{3n}$ (regardless of the distribution of s). As such, there are up to $q_2 + 1$ independent and uniformly random r^* values. The probability that one of $\mathcal{A}$'s q_1 $H_1(\mathsf{pw}^*, r^*)$ query happens *before* the $H_2(\mathsf{pw}^*, T)$ query (when r^* is generated) is upper-bounded by $q_1(q_2 + 1)/2^{3n}$.[2]

Now suppose the aborting condition does not happen. The difference between Game 4 and Game 5 is that Game 4 samples $H_1(\mathsf{pw}^*, r^*)$ uniformly at random, whereas Game 5 defines it as T/pk^*. Note that

[1] A slightly confusing case is when $s^* \neq s \wedge T^* = T$, causing P' to query $t' := H_2(\mathsf{pw}', T)$; and $\mathcal{A}$ also queries $H_2(\mathsf{pw}', T)$. In this case P''s $r' = s^* \oplus t'$ and $\mathcal{A}$'s $r^* = s \oplus t'$ are different, causing P''s values R', pk' and $\mathcal{A}$'s values R^*, pk^* to be independent of each other. Therefore, the game should run P''s algorithm as usual when P' queries $H_2(\mathsf{pw}', T)$ (which generates R', pk'), and use the new method described in Game 5 when $\mathcal{A}$ makes the same query (which generates R^*, pk^*).

[2] P also makes an H_1 query, but it is on (pw, r) which is guaranteed to be different from (pw^*, r^*) (in the case where $\mathcal{A}$ queries $H_2(\mathsf{pw}^*, T)$) and (pw', r') (in the case where P' queries $H_2(\mathsf{pw}', T)$).

- For $H_2(\mathsf{pw}^*, T)$ queries where $\mathsf{pw}^* \neq \mathsf{pw}'$, sk^* (or even pk^*) is not used anywhere in the game;
- For the $H_2(\mathsf{pw}', T)$ query, we cannot say the above anymore, since pk' is used while computing c and K'. However, since $\mathsf{pw} \neq \mathsf{pw}'$, we know that $pk = T/H_1(\mathsf{pw}, r)$ is independent of $pk' = T/H_1(\mathsf{pw}', r')$, so sk' is still not used anywhere else in the game.

So, the distinguishing advantage between Game 4 and Game 5 can be reduced to the UNI-PK property of KEM. We only sketch the reduction $\mathcal{R}_{\text{UNI-PK}}$ here: $\mathcal{R}_{\text{UNI-PK}}$, on input pk^* (which is either an honestly generated public key or a uniformly random value), samples $i \leftarrow [q_2 + 1]$, and answers the (either $\mathcal{A}$'s or P''s) first $i - 1$ H_2 queries as in Game 4 and the last $q_2 - i + 1$ H_2 queries as in Game 5; for the i-th H_2 query, $\mathcal{R}_{\text{UNI-PK}}$ computes

$$r^* := s \oplus t^*, \quad R^* := T/pk^*$$

and programs $H_1(\mathsf{pw}^*, r^*) := R^*$. If this H_2 query is on (pw', T), $\mathcal{R}_{\text{UNI-PK}}$ also uses its input pk^* to compute c and K'. (Note that the $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record is not actually used in Game 5, so $\mathcal{R}_{\text{UNI-PK}}$ does not need to store it.) $\mathcal{R}_{\text{UNI-PK}}$ simulates other parts of Game 4/5 honestly. Finally, $\mathcal{R}_{\text{UNI-PK}}$ copies $\mathcal{Z}$'s output bit.

A standard hybrid argument shows that (assuming the aborting condition does not happen) $\mathcal{Z}$'s distinguishing advantage between Game 4 and Game 5 is upper-bounded by $(q_2 + 1)\mathbf{Adv}^{\text{UNI-PK}}_{\mathcal{R}_{\text{UNI-PK}}}$.[3] Overall, we have

$$\mathbf{Dist}^{4,5}_{\mathcal{Z}} \leq (q_2 + 1)\mathbf{Adv}^{\text{UNI-PK}}_{\mathcal{R}_{\text{UNI-PK}}} + \frac{q_1(q_2 + 1)}{2^{3n}}.$$

This game corresponds to game G_{10} in [ABJ25]. However, there are two differences:

1. *[ABJ25] runs the procedure above to generate pk', r', R' and program H_1 as long as P' makes an $H_2(\mathsf{pw}', T)$ query — that is, including the case where $s^* \neq s \wedge T^* = T$. (If $T^* \neq T$ then of course P' does not make an $H_2(\mathsf{pw}', T)$ query and there is no change from the real world.) The reason for making this change is to prepare for Game 6 and Game 7 below, which simplifies P''s algorithm when $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ (i.e., if we did not make the change in Game 5 first, the reductions in Game 6 and Game 7 would not go through); therefore, it is unnecessary (and potentially confusing) to make this change when $s^* \neq s \wedge T^* = T$.*
2. *The aborting case was ruled out in game G_2 in [ABJ25] (as aborting condition [A2b1]).*

[3] Note that here we cannot make q_2+1 separate reductions to the PR-PK property, and must reduce to PR-PK "in one blow". Furthermore, the calculation of the reduction's advantage is more complicated than the normal case where the reduction wins as long as its guess is correct. This is similar to, say, why composing a PRG polynomial times yields another PRG; see, e.g., [BS23, Section 3.4.3] for details.

Game 6: In the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, sample $SK', \tau' \leftarrow \{0,1\}^n$. ($c$ is still computed as before.)

Let Query_2 be the event that $\mathcal{A}$ queries *either* $H_{\mathsf{key}}(K')$ *or* $H_{\mathsf{tag}}(\mathsf{pw}', pk', c, K')$. Clearly Game 5 and Game 6 are identical unless Query_2 happens. We upper-bound $\Pr[\mathsf{Query}_2]$ via a reduction $\mathcal{R}_{\mathrm{OW1}}$ to the OW property of KEM. This is somewhat similar to the reduction $\mathcal{R}_{\text{OW-1PCA}}$ to the OW-1PCA property in Game 3, and we highlight the differences in blue:

Reduction $\mathcal{R}_{\mathrm{OW1}}$:

Sample $i \leftarrow [q_{\mathsf{key}} + q_{\mathsf{tag}}]$ as a guess that $\mathcal{A}$'s i-th H_{key} or H_{tag} query triggers Query_2, and $j \leftarrow [q_2 + 1]$ as a guess that the j-th H_2 query (by either $\mathcal{A}$ or P') is $H_2(\mathsf{pw}', T)$.
On input (pk', c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T). Send (s, T) to $\mathcal{A}$.
2. Whenever there is an $H_2(\star, T)$ query (by either $\mathcal{A}$ or $\mathcal{R}_{\mathrm{OW1}}$ itself on behalf of P' — see step 3 below), do what's described in Game 5. However, if this is the j-th query, use $\mathcal{R}_{\mathrm{OW1}}$'s input pk' instead of generating pk^* freshly.
3. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise
   (a) If $\mathcal{A}$ has not queried $H_2(\mathsf{pw}', T)$, make this query on behalf of P'.
   (b) Sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\mathrm{OW1}}$'s inputs.)
4. On (c^*, τ^*) from $\mathcal{A}$,
   (a) Find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)
   (b) Compute $K := \mathsf{Decap}(sk, c)$. ($sk$ is the KEM secret key generated in step 1.) // No need to query a plaintext-checking oracle since pk on the P side and pk' on the P' side are independent of each other, so the reduction can sample sk on its own
   If $K = K^*$, output $SK := H_{\mathsf{key}}(K)$ to $\mathcal{Z}$.
   If $K \neq K^*$, output $SK := \perp$ to $\mathcal{Z}$.
5. On $\mathcal{A}$'s i-th H_{key} or H_{tag} query, if it is in the form of $H_{\mathsf{key}}((K')^*)$ or $H_{\mathsf{tag}}(\mathsf{pw}', pk', c, (K')^*)$ for some $(K')^*$, output $(K')^*$; otherwise abort.

Assuming Query_2 happens and $\mathcal{R}_{\mathrm{OW1}}$'s guess of index j is correct, the only difference between Game 5/6 and $\mathcal{R}_{\mathrm{OW1}}$'s simulation (until Query_2 happens) is that $\mathcal{R}_{\mathrm{OW1}}$ uses its input pk' to program $H_1(\mathsf{pw}', r')$, and then uses its input $c \leftarrow \mathsf{Encap}(pk')$ as the P'-to-P message. Since c is also computed as $\mathsf{Encap}(pk')$

in Game 5/6, $\mathcal{R}_{\mathrm{OW1}}$ simulates Game 5/6 perfectly. Furthermore, if $\mathcal{R}_{\mathrm{OW1}}$'s guess of index i is correct then it wins the OW game. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{5,6} \leq \Pr[\mathsf{Query}_2] \leq (q_2+1)(q_{\mathsf{key}}+q_{\mathsf{tag}})\mathbf{Adv}^{\mathrm{OW}}_{\mathcal{R}_{\mathrm{OW1}}}.$$

Remark. It is very subtle why the reduction needs to guess the index j and lose a factor of q_2+1 (and we applaud [ABJ25] for getting it right). The reason is as follows. Consider the following environment $\mathcal{Z}$:

1. Initiate P's session on password pw and receive (s,T).
2. Instruct $\mathcal{A}$ to make a large number of $H_2(\mathsf{pw}^*,T)$ queries (let the result be t^*), one of which is $H_2(\mathsf{pw}',T)$ for a special value pw'.
3. For each of the H_2 queries in step 2,
$$\text{compute } r^* := s \oplus t^*, \quad \text{query } H_1(\mathsf{pw}^*, r^*).$$
4. Initiate P''s session on password pw' and instruct $\mathcal{A}$ to forward (s,T) to P'.

To simulate Game 5/6 correctly, $\mathcal{R}_{\mathrm{OW1}}$ must use its input pk' to program $H_1(\mathsf{pw}',r') := T/pk'$ in step 3. The problem is that $\mathcal{R}_{\mathrm{OW1}}$ *does not know* pw' *until step 4, so it has no idea which* $H_1(\mathsf{pw}',r')$ *query in step 3 needs to be programmed using* pk'! The only way to get around is to make a guess over all H_2 queries, which incurs a q_2+1 loss. (If we require $\mathcal{Z}$ to always initiate P and P''s sessions simultaneously, this loss can be avoided since in step 3 $\mathcal{R}_{\mathrm{OW1}}$ must have already learned pw'.)

This game corresponds to game G_{11} *in [ABJ25], except that they reduce to OW-PCA and avoids the* $q_{\mathsf{key}}+q_{\mathsf{tag}}$ *loss (cf. the comment after Game 3).*

Game 7: Again in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*,T^*) = (s,T)$, recall Game 6 does the following on the P' side: Generate $(pk',\star) \leftarrow \mathsf{Gen}$, query $t' := H_2(\mathsf{pw}',T)$, and compute

$$r' := s \oplus t', \quad R' := T/pk'.$$

((s,T) are the honest P-to-P' message.) If $H_1(\mathsf{pw}',r')$ has already been defined and it is not R', abort. Otherwise program $H_1(\mathsf{pw}',r') := R'$, compute $(c,\star) \leftarrow \mathsf{Encap}(pk')$, and sample $SK', \tau' \leftarrow \{0,1\}^n$. Finally, output SK' and send (c,τ') to P. (Note that K' is not used anywhere since Game 6 — in particular, P' does not use K' to compute SK' or τ' anymore — so we do not need to explicitly mention it.) In this game, we independently generate $(pk'',\star) \leftarrow \mathsf{Gen}$ and compute $(c,\star) \leftarrow \mathsf{Encap}(pk'')$ (the underlined expression).

The difference between Game 6 and Game 7 is that in Game 6 c is generated by

$$pk' = \frac{T}{H_1(\mathsf{pw}', s \oplus H_2(\mathsf{pw}',T))}$$

which $\mathcal{A}$ can compute, whereas in Game 7 c is generated by an independent pk''. We construct a reduction $\mathcal{R}_{\text{ANO}}$ to the ANO property of KEM. This reduction, in fact, is identical to the reduction $\mathcal{R}_{\text{OW1}}$ to the OW property in Game 6 except the last step; for completeness, we still present the full reduction and highlight the difference in blue:

Reduction $\mathcal{R}_{\text{ANO}}$:

Sample $j \leftarrow [q_2 + 1]$ as a guess that the j-th H_2 query (by either $\mathcal{A}$ or P') is $H_2(\mathsf{pw}', T)$.
On input (pk', c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T). Send (s, T) to $\mathcal{A}$.
2. Whenever there is an $H_2(\star, T)$ query (by either $\mathcal{A}$ or $\mathcal{R}_{\text{ANO1}}$ itself on behalf of P' — see step 3 below), do what's described in Game 5. However, if this is the j-th query, use $\mathcal{R}_{\text{ANO}}$'s input pk' instead of generating pk^* freshly.
3. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise
   (a) If $\mathcal{A}$ has not queried $H_2(\mathsf{pw}', T)$, make this query on behalf of P'.
   (b) Sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{ANO}}$'s inputs.)
4. On (c^*, τ^*) from $\mathcal{A}$,
   (a) Find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)
   (b) Compute $K := \mathsf{Decap}(sk, c^*)$. ($sk$ is the KEM secret key generated in step 1.) // No need to query a plaintext-checking oracle since pk on the P side and pk' on the P' side are independent of each other, so the reduction can sample sk on its own
   If $K = K^*$, output $SK := H_{\mathsf{key}}(K)$ to $\mathcal{Z}$.
   If $K \neq K^*$, output $SK := \perp$ to $\mathcal{Z}$.
5. When $\mathcal{Z}$ outputs bit b^*, copy $\mathcal{Z}$'s output.

Assume $\mathcal{R}_{\text{ANO}}$'s guess of index j is correct. If $b = 0$ then $c \leftarrow \mathsf{Encap}(pk')$, i.e., the computation of R' and c use the same pk', so $\mathcal{R}_{\text{ANO}}$ simulates Game 6; if $b = 1$ then $c \leftarrow \mathsf{Encap}(pk'')$, i.e., the computation of c uses an independently generated pk'', so $\mathcal{R}_{\text{ANO}}$ simulates Game 7. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{6,7} \leq (q_2 + 1)\mathbf{Adv}_{\mathcal{R}_{\text{ANO}}}^{\text{ANO}}.$$

Game 8: Again in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, we outright skip the computation of t', r', R' and the programming of H_1 (the brown text

in Game 7). (Since pk' is not generated anymore, we can rename pk'' — the KEM public key from which c is computed — to pk'.)

Note that the aborting condition, which was originally introduced in Game 5, is removed (for the specific $H_2(\mathsf{pw}', T)$ query by P'). By an analysis similar to that in Game 5, the aborting probability in Game 7 is upper-bounded by $q_1/2^{3n}$. If the aborting condition does not happen, the brown text is independent of the rest of the game, so removing it does not affect the distribution of $\mathcal{Z}$'s output. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{6,7} \leq \frac{q_1}{2^{3n}}.$$

The two games above combined correspond to game G_{12} in [ABJ25]. [ABJ25] does not have the intermediate Game 7 in our proof, and outright removes the brown text from what's our Game 6. We believe it is clearer to include our Game 7, where H_2 is still queried and H_1 is still programmed: in this way it is easier to see why the reduction to ANO simulates the games correctly. (Of course, it is possible that an experienced cryptographer can see through the two game hops in one shot, but the author of this paper admits that he is not that experienced and has to work out the game hops and reductions the "dumb" way.) Also, [ABJ25] reduces to ANO-PCA[4], although it also says that the plaintext-checking oracle is not needed.

Computation of the P-to-P' message.

Game 9: Note that up to now we have not changed P's algorithm before sending (s, T) (in particular, Game 5 dealt with H_2 queries by $\mathcal{A}$ and P', but not P). In this game we modify it as follows: Sample $s \leftarrow \{0,1\}^{3n}, T \leftarrow \mathbb{G}$. If $\mathcal{A}$ has queried $H_2(\star, T)$, abort. Otherwise query $t := H_2(\mathsf{pw}, T)$, generate $(pk, sk) \leftarrow \mathsf{Gen}$, and compute

$$r := s \oplus t, \quad R := T/pk.$$

If $H_1(\mathsf{pw}, r)$ has already been defined and it is not R, abort. Otherwise program $H_1(\mathsf{pw}, r) := R$.

We first upper-bound the aborting probability. Apart from the one made by P, there are q_2 H_2 queries in total (note that P' does not query H_2 anymore since Game 8). Thus, the probability that one of the query inputs happens to contain T is upper-bounded by $q_2/|\mathbb{G}|$. By an analysis similar to that in Game 5 and Game 8, the probability of the second aborting event is upper-bounded by $q_1/2^{3n}$.

Now suppose neither of the aborting events happens. The following equations hold in both Game 8 and Game 9:

$$r = s \oplus H_2(\mathsf{pw}, T), \quad T = R \cdot pk.$$

Here, pk is generated by Gen, and $H_2(\mathsf{pw}, T)$ is fixed by an H_2 query once T is fixed. The four remaining variables r, s, R, T are constrained by two equations and have $4 - 2 = 2$ degrees of freedom: if we sample $r \leftarrow \{0,1\}^{3n}, R \leftarrow \mathbb{G}$ then

[4] [ABJ25] actually mentions ANO-CPA, which appears to be a typo.

we get Game 8, and if we sample $s \leftarrow \{0,1\}^{3n}, T \leftarrow \mathbb{G}$ then we get Game 9. Thus, Game 8 and Game 9 are identical. We have

$$\mathbf{Dist}^{8,9}_{\mathcal{Z}} \leq \frac{q_1}{2^{3n}} + \frac{q_2}{|\mathbb{G}|}.$$

The game above corresponds to game G_{13} in [ABJ25], with one difference: our game aborts if $\mathcal{A}$ has queried $H_2(\mathsf{pw}^*, T)$ for any pw^* when T is sampled, whereas the game in [ABJ25] aborts only if $\mathcal{A}$ has queried $H_2(\mathsf{pw}, T)$. Note that the simulator does not know pw, so it cannot tell if $\mathcal{A}$ has queried $H_2(\mathsf{pw}, T)$ and instead has to abort as long as $\mathcal{A}$ has queried $H_2(\star, T)$. (The simulator in [ABJ25] is correct, but its game hops do not match the ideal world regarding which H_2 queries will trigger an abortion.) Also, we believe our argument under this game is much more concise and cleaner (our presentation is, in spirit, similar to [JRX25, Lemma 4.5, Hybrid 2]).

Game 10: When P's session begins, halt after checking if $H_2(\star, T)$ has been defined (i.e., before the brown text in Game 9). Then when P receives (c^*, τ^*), if $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, do what's described in Game 1. Otherwise,

- If $H_2(\mathsf{pw}, T)$ has now been defined (via a query by $\mathcal{A}$), there must be a corresponding record $\langle \mathsf{pw}, pk, sk \rangle$ (see Game 5).
- If $H_2(\mathsf{pw}, T)$ is still undefined, run the brown text in Game 9.

Either way, pk, sk are defined and the game can proceed as usual.

The only difference between Game 9 and Game 10 lies in the fact that when we move the brown text around, the game might abort — as part of the brown text — at different times. Since the aborting event happens with probability at most $q_1/2^{3n}$, we have

$$\mathbf{Dist}^{9,10}_{\mathcal{Z}} \leq \frac{q_1}{2^{3n}}.$$

The game above corresponds to game G_{14} in [ABJ25]. Note that we have a $q_1/2^{3n}$ loss in both Game 9 and Game 10; if we remove Game 9 and directly jump to Game 10 from Game 8, this loss does not need to be repeated (since Game 8 and Game 10 are identical unless one of the two aborting events happens). We (as well as [ABJ25]) choose to include Game 9 for clarity.

Intermission. Let's pause a bit and review the changes we have made so far:

1. *P-to-P' message:* Game 10 outright samples (s, T) at random. [changed in Game 9]
2. *P''s session key and P'-to-P message:* On (s^*, T^*) from $\mathcal{A}$ to P', if $(s^*, T^*) = (s, T)$, Game 10 outright samples SK', τ' at random. Regarding c, Game 10 generates $(pk', \star) \leftarrow \mathsf{Gen}$ and computes $(c, \star) \leftarrow \mathsf{Encap}(pk')$. (If $(s^*, T^*) \neq (s, T)$ then there is no change from the real world.) [changed in Games 3, 4, 6, 7, 8]

3. *P's session key:* On (c^*, τ^*) from $\mathcal{A}$ to P, we have made the following changes:
   (a) If $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, Game 10 outright sets $SK := SK'$ if $\tau^* = \tau'$ and $SK := \perp$ otherwise. [changed in Game 1]
   (b) Otherwise if $\mathcal{A}$ makes no $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ resulting in τ^*, or more than one such query, Game 10 outright sets $SK := \perp$. [changed in Game 2]
   (c) Otherwise if sk has not been defined yet, Game 10 queries $H_2(\mathsf{pw}, T)$ on behalf of P, which generates sk (see below). The remaining part is unchanged from the real world.
4. *H_2 queries:* When either $\mathcal{A}$ or P makes a fresh query $H_2(\mathsf{pw}^*, T)$ (let the result be t^*), Game 10 runs the following procedure: Generate $(pk^*, sk^*) \leftarrow \mathsf{Gen}$, and compute

$$r^* := s \oplus t^*, \quad R^* := T/pk^*.$$

   If $H_1(\mathsf{pw}^*, r^*)$ has already been defined and it is not R^*, abort. Otherwise program $H_1(\mathsf{pw}^*, r^*) := R^*$. [changed in Games 5, 10]

See Figure 4 for a summary of Game 10.

Password extraction from P'-to-P message.

Game 11: In the case where $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, if there is no $\langle \mathsf{pw}, \star, \star \rangle$ record, set $SK := \perp$ (i.e., replace the brown text in Figure 4 with "$SK := \perp$").

Game 10 and Game 11 differ only if $\mathcal{A}$ does not make an $H_2(\mathsf{pw}, T)$ query, but makes an $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query resulting in τ^* (for c^*, K^*, τ^* of $\mathcal{A}$'s choice). This in particular requires $\mathcal{A}$ to know pk, which is freshly generated by Gen. Therefore, the probability that (even a computationally unbounded) $\mathcal{A}$ can predict pk is upper-bounded by $H_\infty(pk : (pk, \star) \leftarrow \mathsf{Gen})$. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{10,11} \leq H_\infty(pk : (pk, \star) \leftarrow \mathsf{Gen}).$$

The game above corresponds to game G_{15} in [ABJ25]. [ABJ25] does not include the condition $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, i.e., it appears that this game rewrites what our Game 2 does, which is incorrect.

The argument in this game critically relies on pk being included in H_{tag} inputs.

Game 12: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c \wedge \tau^* \neq \tau'$, do what's in the $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$ case.

In Game 11 SK is outright set to $\perp$, whereas here it is treated the same as the "normal" case. Let Query_3 be the event that $\mathcal{A}$ queries $H_{\mathsf{tag}}(\mathsf{pw}, pk, c, K)$, where pk is the KEM public key generated on $\mathcal{A}$'s $H_2(\mathsf{pw}, T)$ query; Game 11 and Game 12 are identical unless Query_3 happens. To trigger Query_3, $\mathcal{A}$ must

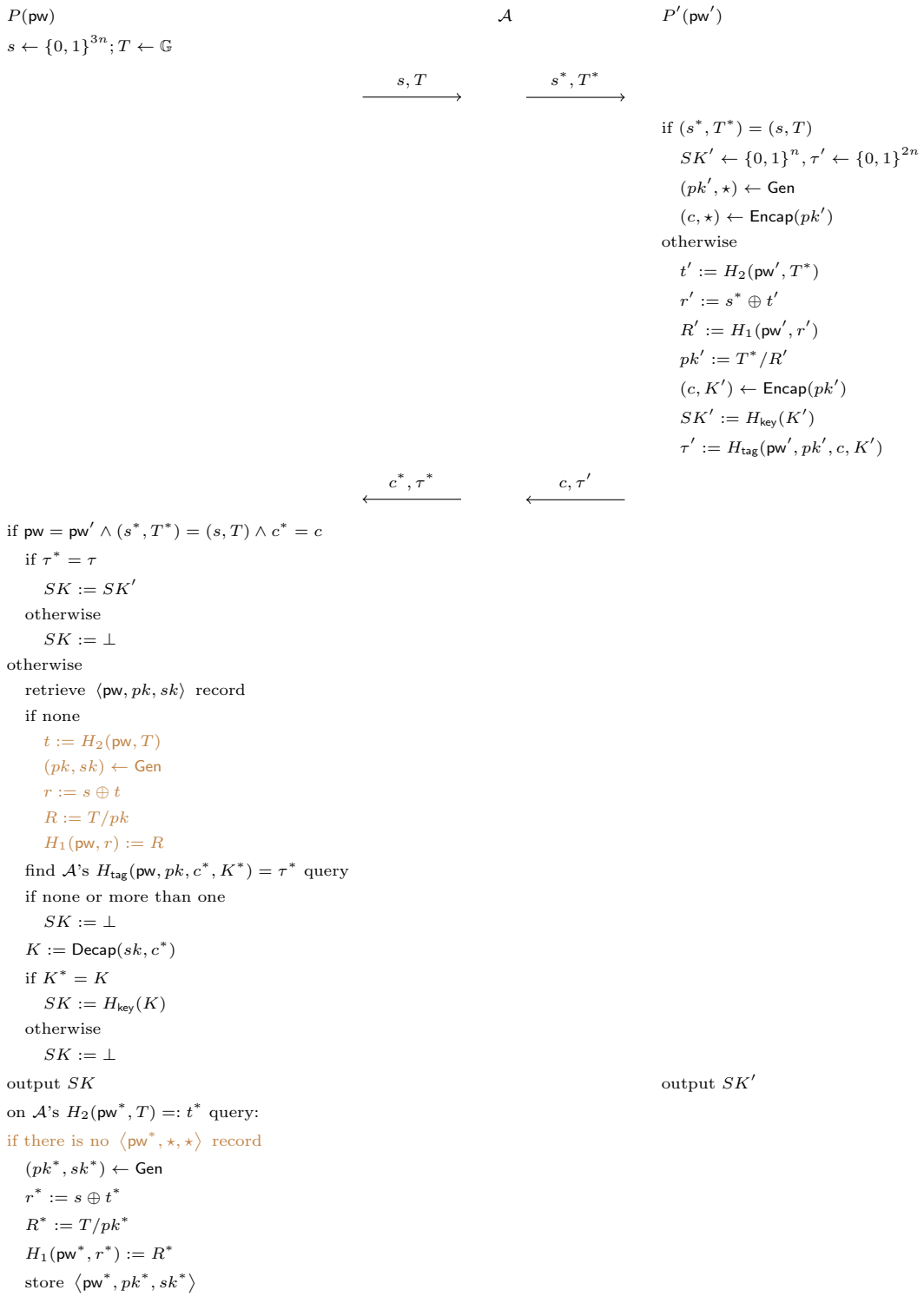

Figure 4: Game 10 (with various aborting conditions omitted)

know K, which suggests a reduction $\mathcal{R}_{\mathrm{OW2}}$ to the OW property of KEM. This reduction is simple, so we only provide a sketch: $\mathcal{R}_{\mathrm{OW2}}$, on inputs (pk, c), invokes $\mathcal{Z}$ and simulates (s, T) and the P' side honestly. (Note that P' uses a freshly generated pk' independent of the P side.) When $\mathcal{A}$ queries $H_2(\mathsf{pw}, T)$, $\mathcal{R}_{\mathrm{OW2}}$ uses its input pk in lieu of $(pk, \star) \leftarrow \mathsf{Gen}$. Along the way, as long as $\mathcal{R}_{\mathrm{OW2}}$ detects that $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c \wedge \tau^* \neq \tau'$ does not happen, it aborts. Finally, $\mathcal{R}_{\mathrm{OW2}}$ finds the (unique) $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c, K^*)$ query resulting in τ^*, and outputs K^*. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{11,12} \leq \Pr[\mathsf{Query}_3] \leq \mathbf{Adv}^{\mathrm{OW}}_{\mathcal{R}_{\mathrm{OW2}}}.$$

The game above is missing in [ABJ25]. Note that in the ideal world, the simulator cannot tell if $\mathsf{pw} = \mathsf{pw}'$*, so when it sees* $(s^*, T^*) = (s, T) \wedge c^* = c \wedge \tau^* \neq \tau'$*, it has to always try to extract a password guess; hence, the game above has to be here. (In the* $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c \wedge \tau^* \neq \tau'$ *case, we first outright set* $SK := \perp$ *in Game 1 and then "walk back" in Game 12, since the change in Game 1 is necessary for the reductions in Game 3 and Game 4 to go through.)*

Game 13: In the case where $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau))$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, c^*, K^*)$ query whose result is τ^*. (The difference is that Game 12 only goes over all of $\mathcal{A}$'s queries in the form of $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$, whereas this game goes over all of $\mathcal{A}$'s queries in the form of $H_{\mathsf{tag}}(\star, \star, c^*, \star)$.) If there is none or more than one, set $SK := \perp$. Otherwise there is a unique such $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, c^*, K^*)$ query, and

- If $\mathsf{pw}^* \neq \mathsf{pw}$, set $SK := \perp$.
- If $\mathsf{pw}^* = \mathsf{pw}$, run the remaining part of P''s algorithm except that we now use pw^* instead of pw — that is, if there is no $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record (note that the pk^* here has to match the pk^* in $\mathcal{A}$'s H_{tag} query) then set $SK := \perp$; otherwise compute $K := \mathsf{Decap}(sk^*, c^*)$, and if $K^* = K$ then set $SK := H_{\mathsf{key}}(K)$, otherwise set $SK := \perp$.

If $\mathcal{A}$ makes no $H_{\mathsf{tag}}(\star, \star, c^*, \star)$ query resulting in τ^*, then in particular $\mathcal{A}$ makes no $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ query resulting in τ^*, so both Game 12 and Game 13 set $SK := \perp$. The probability that $\mathcal{A}$ makes two different $H_{\mathsf{tag}}(\star, \star, c^*, \star)$ queries both resulting in τ^* is upper-bounded by $q_{\mathsf{tag}}(q_{\mathsf{tag}} - 1)/2^{2n+1}$. Now suppose $\mathcal{A}$ makes a unique $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, c^*, K^*)$ query resulting in τ^*:

- If $\mathsf{pw}^* \neq \mathsf{pw}$, then $\mathcal{A}$ does not make a $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ query resulting in τ^* (because there is only one $H_{\mathsf{tag}}(\star, \star, c^*, \star)$ query resulting in τ^*, and it is on pw^*), so both Game 12 and Game 13 set $SK := \perp$.
- If $\mathsf{pw}^* = \mathsf{pw}$, then there is no difference between Game 12 (which uses pw) and Game 13 (which uses pw^*).

We conclude that

$$\mathbf{Dist}_{\mathcal{Z}}^{12,13} \leq \frac{q_{\mathsf{tag}}(q_{\mathsf{tag}} - 1)}{2^{2n+1}}.$$

The game above is also missing in [ABJ25]. The H_{tag} collision case was ruled out in game G_2 in [ABJ25] (as aborting condition [A1])[5]; however, conceptually speaking there still needs to be such a bridge game, to make sure that the game hops match the ideal world at the end.

The definition of this game, together with its argument, critically rely on pw being included in H_{tag} inputs.

Game 14: In the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, set $SK := \perp$.

In this case $\tau^* = \tau'$ is uniformly random and independent of the rest of the game, so the probability that $\mathcal{A}$ makes an H_{tag} query resulting in τ^* is at most $q_{\mathsf{tag}}/2^{2n}$. If $\mathcal{A}$ does not make such a query, then both Game 13 and Game 14 set $SK := \perp$. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{13,14} \leq \frac{q_{\mathsf{tag}}}{2^{2n}}.$$

The game above is also missing in [ABJ25]. Again in the ideal world, the simulator cannot tell if $\mathsf{pw} = \mathsf{pw}'$, so when it sees $\mathcal{A}$ is passive, it has to let the PAKE functionality check if $\mathsf{pw} = \mathsf{pw}'$ and set SK to be equal to SK' or $\perp$ accordingly; hence, the game above has to be here.

Password extraction from P-to-P' message.

Game 15: In the case where $(s^*, T^*) \neq (s, T)$, find $\mathcal{A}$'s $H_2(\mathsf{pw}^*, T^*)$ query[6] (let the result be t^*) that satisfies *either* of the following:

1. $\mathcal{A}$ queried $H_1(\mathsf{pw}^*, r^*)$ *before* the $H_2(\mathsf{pw}^*, T^*)$ query, where $r^* = s^* \oplus t^*$.
2. $\mathcal{A}$ also queried $H_2(\mathsf{pw}^*, T)$, and

$$s \oplus s^* = H_2(\mathsf{pw}^*, T) \oplus t^*.$$[7]

(This $H_2(\mathsf{pw}^*, T^*)$ query is what's called the "anchor query" in [JRX25].) If there is more than one such query, abort.

The aborting condition is met only in the following three cases:

- There is a collision in type (1) above, i.e., there are $r^*, t^*, (r^*)', (t^*)'$ such that
$$r^* \oplus t^* = (r^*)' \oplus (t^*)'$$

[5] In our game hops, we ruled out H_{tag} collisions for different K^* values in Game 2, and then additionally rule out H_{tag} collisions for different $(\mathsf{pw}^*, pk^*, K^*)$ pairs in Game 13. We could have done both in Game 2 and avoided the additional loss here; however, we believe the current approach is clearer.

[6] The notations are slightly messed up, since the pw^* here has nothing to do with what's called pw^* in Game 13. Essentially, the pw^* here is $\mathcal{A}$'s guess of P''s password (which is called $(\mathsf{pw}')^*$ in the simulator), whereas the pw^* in Game 13 is $\mathcal{A}$'s guess of P's password. The good news is that the pw^* in Game 13 is never reused once Game 13 is done, so there is no potential confusion.

[7] Note that in this case $T^* \neq T$ since otherwise $T^* = T \Rightarrow H_2(\mathsf{pw}^*, T) = t^* \Rightarrow s^* = s \oplus H_2(\mathsf{pw}^*, T) \oplus t^* = s$, so $(s^*, T^*) = (s, T)$ which violates the condition of this game. (A very similar argument was made in the proof of Claim 1 in Game 2.)

(so $\mathcal{A}$ can set s^* to be this value). Note that all four values in the equation are determined by two (different) H_1 queries and one H_2 query, and for each set of these queries, there is a $1/2^{3n}$ chance that the equation will hold. Therefore, the probability of this case is upper-bounded by $q_1(q_1-1)q_2/2^{3n}$.[8]

- There is a collision in type (2) above, i.e., there are $\mathsf{pw}^*, t^*, (\mathsf{pw}^*)', (t^*)'$ such that
$$H_2(\mathsf{pw}^*, T) \oplus t^* = H_2((\mathsf{pw}^*)', T) \oplus (t^*)'$$
(so $\mathcal{A}$ can set s^* to be $s \oplus H_2(\mathsf{pw}^*, T) \oplus t^*$). Note that all four values in the equation are determined by three (different) H_2 queries $H_2(\mathsf{pw}^*, T)$, $H_2((\mathsf{pw}^*)', T)$ and $H_2(\mathsf{pw}^*, T^*)$; by an argument similar to above, the probability of this case is upper-bounded by $q_2(q_2-1)(q_2-2)/2^{3n}$.

- There is a collision between type (1) and type (2), i.e., there are $r^*, t^*, (\mathsf{pw}^*)'$, $(t^*)'$ such that
$$r^* \oplus t^* = s \oplus H_2((\mathsf{pw}^*)', T) \oplus (t^*)'$$
(so $\mathcal{A}$ can set s^* to be this value). Note that s is already fixed, and all four other values in the equation are determined by two (different) H_2 queries and one H_1 query; by an argument similar to above, the probability of this case is upper-bounded by $q_1 q_2(q_2-1)/2^{3n}$.

Summing up, we have

$$\mathbf{Dist}^{14,15}_{\mathcal{Z}} \le \frac{q_2[q_1(q_1+q_2-2)+(q_2-1)(q_2-2)]}{2^{3n}}.$$

This game corresponds to part of game G_2 in [ABJ25] (as aborting conditions [A2a] and [A2b2]). The game in [ABJ25] aborts more often than necessary, though: every time $\mathcal{A}$ makes an $H_2(\star,\star)$ query, the game performs the checks and aborts accordingly, while it is only necessary to go over all of $\mathcal{A}$'s $H_2(\star, T^*)$ queries for the specific T^* as part of $\mathcal{A}$'s message to P'.

Performing the checks on $\mathcal{A}$'s H_2 query, instead of when $\mathcal{A}$ sends the (s^*, T^*) message, also adds more bookkeeping and complicates the proof. In [ABJ25], after performing the checks, the game (if it does not abort) computes the corresponding KEM public key and stores it together with (pw^*, T^*) (the record is called ForwS); later when $\mathcal{A}$ sends (s^*, T^*), the game first fetches the record to recover pw^*, and if $\mathsf{pw}^* = \mathsf{pw}'$, the game then recovers the KEM public key and uses it to complete the remaining part of P''s algorithm. This creates a conceptual discrepancy with the ideal world, and a game hop (game G_{16} in [ABJ25]) is needed. By performing the checks when $\mathcal{A}$ sends the (s^*, T^*), we avoid all these and skip their game G_{16}.

[8] A sloppier argument would say that there are q_1 possible r^* values and q_2 possible t^* values, so there are $q_1 q_2$ possible $r^* \oplus t^*$ values and the probability of a collision is upper-bounded by $q_1 q_2(q_1 q_2 - 1)/2^{3n+1}$. This yields a looser bound since those $r^* \oplus t^*$ values are not all independent of each other. The same goes for the two cases below.

Correct password guess for P'.

Game 16: Again in the case where $(s^*, T^*) \neq (s, T)$, if pw^* is defined in type (2) in Game 15 and $\mathsf{pw}^* = \mathsf{pw}'$, retrieve record $\langle \mathsf{pw}^*, pk^*, \star \rangle$ (there must be one since such a record is created when $\mathcal{A}$ queries $H_2(\mathsf{pw}^*, T)$, as per Game 5) and set $pk' := pk^* \cdot (T^*/T)$.[9]

In Game 15, we have (in P''s algorithm)

$$pk' = \frac{T^*}{R'} = \frac{T^*}{H_1(\mathsf{pw}', r')} = \frac{T^*}{H_1(\mathsf{pw}', s^* \oplus t')} = \frac{T^*}{H_1(\mathsf{pw}', s^* \oplus H_2(\mathsf{pw}', T^*))},$$

but also (in the creation of the record $\langle \mathsf{pw}^*, pk^*, \star \rangle$)

$$pk^* = \frac{T}{R^*} = \frac{T}{H_1(\mathsf{pw}^*, r^*)} = \frac{T}{H_1(\mathsf{pw}^*, s \oplus t^*)} = \frac{T}{H_1(\mathsf{pw}^*, s \oplus H_2(\mathsf{pw}^*, T))}.$$

The condition of this game specifies that $\mathsf{pw}^* = \mathsf{pw}'$, and type (2) in Game 15 specifies that $s \oplus H_2(\mathsf{pw}^*, T) = s \oplus H_2(\mathsf{pw}^*, T^*)$. This means that the denominators in the expressions of pk' and pk^* are equal, and thus $pk' = pk^* \cdot (T^*/T)$ — exactly as in Game 15. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{15,16} = 0.$$

This game corresponds to game G_{17} in [ABJ25].

Incorrect password guess for P'.

Game 17: Again in the case where $(s^*, T^*) \neq (s, T)$, if $\mathsf{pw}^* \neq \mathsf{pw}'$ or no pw^* is defined, sample $pk' \leftarrow \mathbb{G}$, compute $(c, \star) \leftarrow \mathsf{Encap}(pk')$, and sample $SK', \tau' \leftarrow \{0,1\}^n$.

We construct a reduction $\mathcal{R}_{\text{UNI-ANO-RO}}$ to the $(q_1+1)(q_2+1)$-UNI-ANO-RO property of KEM:

Reduction $\mathcal{R}_{\text{UNI-ANO-RO}}$:

On inputs $(pk_{1,1}, \ldots, pk_{q_1+1,q_2+1})$,

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, sample $s \leftarrow \{0,1\}^{3n}, T \leftarrow \mathbb{G}$. Send (s, T) to $\mathcal{A}$.

2. Whenever there is an $H_2(\star, T)$ query by $\mathcal{A}$, do what's described in Game 5. Furthermore, for the j-th query to H_2 (by $\mathcal{A}$ or $\mathcal{R}_{\text{UNI-ANO-RO}}$ itself on behalf of P' — see step 3 below), say it is $H_2(\mathsf{pw}^*, T^*) =: t^*$, and the i-th H_1 query in the form of $H_1(\mathsf{pw}^*, r^*)$, compute

$$s^* := r^* \oplus t^*, \quad R^* := T^*/pk_{i,j}.$$

[9] What's called pk^* here is $(pk')^*$ in the simulator, as there is another pk^* on the P side as defined in Game 13 (cf. Footnote 6).

(a) If $H_1(\mathsf{pw}^*, r^*)$ has been defined when the $H_2(\mathsf{pw}^*, T^*)$ query happens (either because it was queried *before* the $H_2(\mathsf{pw}^*, T^*) =: t^*$ query or because it has been programmed by $\mathcal{R}_{\text{UNI-ANO-RO}}$ while answering $\mathcal{A}$'s $H_2(\mathsf{pw}^*, T)$ query), store $\langle \mathsf{PasswordGuess}, \mathsf{pw}^*, s^*, T^* \rangle$. // Anchor query, i.e., pw^* is the password guess corresponding to (s^*, T^*)
At any time, if there is more than one $\langle \star, s^*, T^* \rangle$ record with the same s^*, T^*, abort.

(b) Otherwise program $H_1(\mathsf{pw}^*, r^*) := R^*$.

3. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}',$ "respondent") from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $(s^*, T^*) = (s, T)$, output 1. If there is a record $\langle \mathsf{PasswordGuess}, \mathsf{pw}', s^*, T^* \rangle$, also output 1. Otherwise // Incorrect or no password guess (the case that matters in this game hop)

(a) If $\mathcal{A}$ has not queried $H_2(\mathsf{pw}', T^*)$ or $H_1(\mathsf{pw}', r')$ (where $r' = s^* \oplus H_2(\mathsf{pw}', T^*)$), make these queries on behalf of P'.

(b) Say $H_2(\mathsf{pw}', T^*)$ is the j^*-th query to H_2 and $H_1(\mathsf{pw}', r')$ is the i^*-th query to H_1 in the form of $H_1(\mathsf{pw}', \star)$. Output (i^*, j^*) to the challenger and receive (c, SK', τ').

(c) Output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$.

4. On (c^*, τ^*) from $\mathcal{A}$,

(a) Retrieve record $\langle \mathsf{pw}, pk, sk \rangle$. If there is no such record, output $SK := \perp$ to $\mathcal{Z}$.

(b) Find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$.

(c) Compute $K := \mathsf{Decap}(sk, c^*)$. ($sk$ is the KEM secret key generated in step 1.) // No need to query a plaintext-checking oracle since pk on the P side and pk' on the P' side are independent of each other, so the reduction can sample sk on its own
If $K = K^*$, output $SK := H_{\mathsf{key}}(K)$ to $\mathcal{Z}$.
If $K \neq K^*$, output $SK := \perp$ to $\mathcal{Z}$.

5. When $\mathcal{Z}$ copies a bit b^*, copy $\mathcal{Z}$'s output.

We first argue that the pw^* as in record $\langle \mathsf{PasswordGuess}, \mathsf{pw}^*, s^*, T^* \rangle$ is exactly the same as the pw^* extracted from (s^*, T^*) in Game 15:

- If $\mathcal{A}$ queried $H_1(\mathsf{pw}^*, r^*)$ before the $H_2(\mathsf{pw}^*, T^*)$ query, this is exactly type (1) in Game 15.
- If $\mathcal{A}$ queried $H_2(\mathsf{pw}^*, T)$, which triggered the programming of $H_1(\mathsf{pw}^*, r^*)$ before the $H_2(\mathsf{pw}^*, T^*)$ query, note that the H_1 instance that is programmed on is $(\mathsf{pw}^*, s \oplus H_2(\mathsf{pw}^*, T))$. Thus,

$$s \oplus H_2(\mathsf{pw}^*, T) = r^* = s^* \oplus H_2(\mathsf{pw}^*, T^*),$$

which is exactly type (2) in Game 15.

Therefore, $\mathcal{R}_{\text{UNI-ANO-RO}}$ accurately decides whether the condition "$\mathsf{pw}^* \neq \mathsf{pw}'$ or no pw^* is defined" is satisfied or not. Next, we argue that $\mathcal{R}_{\text{UNI-ANO-RO}}$ simulates Game 16/17 correctly:

- Suppose $\mathcal{R}_{\text{UNI-ANO-RO}}$ is in the "real" game where $(c, K') \leftarrow \mathsf{Encap}(pk_{i^*,j^*})$ and $SK' = H_{\mathsf{key}}(K'), \tau' = H_{\mathsf{tag}}(\ldots, K')$. For the "special" $H_2(\mathsf{pw}', T^*)$ query and $H_1(\mathsf{pw}', r')$ query, in both Game 16 and $\mathcal{R}_{\text{UNI-ANO-RO}}$'s simulation,
$$s^* = r' \oplus H_2(\mathsf{pw}', T^*), \quad t^* = pk' \cdot H_1(\mathsf{pw}', r');$$
if we sample $H_1(\mathsf{pw}', r') \leftarrow \mathbb{G}$ then we get Game 16, whereas if we sample $pk' \leftarrow \mathbb{G}$ (as pk_{i^*,j^*}) then we get $\mathcal{R}_{\text{UNI-ANO-RO}}$'s simulation. Later, pk' is used to compute c, K' in both Game 16 and $\mathcal{R}_{\text{UNI-ANO-RO}}$'s simulation. Regarding other $H_2(\mathsf{pw}^*, T^*)$ and $H_1(\mathsf{pw}^*, r^*)$ queries, the $pk_{i,j}$ that $\mathcal{R}_{\text{UNI-ANO-RO}}$ uses while programming $H_1(\mathsf{pw}^*, r^*)$ is uniformly random and independent of the rest of the game, so $H_1(\mathsf{pw}^*, r^*)$ is programmed to a uniformly random value — again the same as Game 16.

- Suppose $\mathcal{R}_{\text{UNI-ANO-RO}}$ is in the "random" game where $(c, K') \leftarrow \mathsf{Encap}(pk')$ and $SK' = H_{\mathsf{key}}(K'), \tau' = H_{\mathsf{tag}}(\ldots, K')$ for a freshly sampled pk'. *All* $pk_{i,j}$ values that $\mathcal{R}_{\text{UNI-ANO-RO}}$ uses while programming H_1 are uniformly random and independent of the rest of the game, so H_1 is always programmed to a uniformly random value — the same as Game 17. Later, a freshly sampled pk' is used to compute c, K' in both Game 17 and $\mathcal{R}_{\text{UNI-ANO-RO}}$'s simulation.

We have
$$\mathbf{Dist}^{16,17}_{\mathcal{Z}} \leq \mathbf{Adv}^{(q_1+1)(q_2+1)\text{-UNI-ANO-RO}}_{\mathcal{R}_{\text{UNI-ANO-RO}}}.$$

This game corresponds to games G_{18} and G_{19} in [ABJ25]. [ABJ25] has two games: G_{18} that changes SK' and τ' (via a reduction to the OW-PCA property of KEM), and G_{19} that changes c (via a reduction to the ANO-PCA property of KEM). While we believe their argument is fundamentally correct, it could be significantly improved:

1. *[ABJ25] reduces to OW-PCA and ANO-PCA[10], which is unnecessary: OW and ANO suffice.*
2. *Their simulation strategy is slightly different from ours: they use pk' generated by Gen, rather than sampled uniformly at random. Note that in this "incorrect password guess" case, in the real world $pk' = T^*/H_1(\mathsf{pw}', r')$ is already uniformly random, so this is fundamentally a KEM property about encapsulation on uniformly random (rather than honestly generated) public keys and we believe it is clearer to present it in this way.*[11]

[10] [ABJ25] actually mentions ANO-CPA, which appears to be a typo.

[11] Note that pk' has to be generated honestly in the $(s^*, T^*) = (s, T)$ case, which is our Games 3, 4, 6 and 7: If $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ then sk corresponding to $pk = pk'$ is used on the P side (in decapsulation), so pk' cannot be changed to uniformly random; and the $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ case cannot be separated from the $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ case (since the simulator never knows if $\mathsf{pw} = \mathsf{pw}'$ or not), so pk' cannot be changed to uniformly random in this case either.

3. *Their honest generation of pk' also means that their reductions cannot simulate the games perfectly, since they program $H_1(\mathsf{pw}', r')$ using pk' which is not uniformly random — while it "should" be uniformly random. As such, they have do a reduction to UNI-PK within the OW-PCA/ANO-PCA reduction.*

4. *What makes things even worse is that none of their (or our) reductions knows which of the $H_1(\mathsf{pw}', r')$ queries should be programmed using pk', since $\mathcal{A}$ can make many decryption efforts, resulting in many H_1 values to be programmed,* before *sending (s^*, T^*) to P' (cf. discussion after Definition 1). This means that their UNI-PK reduction cannot be moved "out of" the OW-PCA/ANO-PCA reduction by defining a separate game that uses honestly generated pk' to program $H_1(\mathsf{pw}', r')$, as that would require the* game *to make a guess! What's going on here is essentially a hybrid argument (in the sense of [BS23, Section 3.4.3]), which is not explained clearly in [ABJ25].*

5. *Finally, the reductions in [ABJ25] appear to never check for the "related key attack", i.e., it is unclear what will go wrong if we remove what's our Game 16 (or their game G_{17}).*

We believe it is much cleaner to define a tailored KEM property that allows a single *reduction to go through, which is UNI-ANO-RO. In this way, we can separate out the multiple reductions to basic KEM properties, as well as the hybrid argument, on the level of KEM; this makes our OEKE security argument easier to understand.*[12]

Summary. We summarize the changes since Game 11 (i.e., after Figure 4):

1. *P''s session key and P'-to-P message:* On (s^*, T^*) from $\mathcal{A}$ to P', if $(s^*, T^*) \neq (s, T)$, Game 17 extracts $(\mathsf{pw}')^*$ from (s^*, T^*) using the method in Game 15. Then
   (a) If $(\mathsf{pw}')^* \neq \mathsf{pw}'$, then Game 17 outright samples SK', τ' at random. Regarding c, Game 10 samples $pk' \leftarrow \mathbb{G}$ and computes $(c, \star) \leftarrow \mathsf{Encap}(pk')$. [changed in Game 17]
   (b) If $(\mathsf{pw}')^* = \mathsf{pw}'$, Game 15 specifies two types of extraction of $(\mathsf{pw}')^*$. There is no change from the real world for type (1). For type (2), Game 17 computes $pk' := pk^* \cdot (T^*/T)$ (and uses pk' to compute c', SK', τ'). [changed in Game 16]

2. *P's session key:* On (c^*, τ^*) from $\mathcal{A}$ to P, we have made the following changes:

[12] In principle, we could also combine Game 3 and Game 4, as well as Game 6 and Game 7, via a single reduction to some "integrated" KEM properties. We choose not to do it as having many new, tailored KEM properties could be confusing.

(a) If $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, Game 17 outright sets $SK := \perp$. [changed in Game 14]

(b) If $\neg((s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau'))$, Game 17 retrieves two records:

- $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, or if there is a unique such query with $\mathsf{pw}^* \neq \mathsf{pw}$, Game 17 sets $SK := \perp$.
- $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record stored when $\mathcal{A}$ queries $H_2(\mathsf{pw}^*, T)$. If there is none, Game 17 sets $SK := \perp$.

(Note that pw^* and pk^* in these two records has to match each other.) After that, Game 17 checks if $K^* = \mathsf{Decap}(sk^*, c^*)$, and computes SK using K^* or sets $SK := \perp$ accordingly. [changed in Games 11, 12, 13]

See Figure 5 for a summary of Game 17.

Comparison with the ideal world. We now claim that Game 17 is identical to the ideal world, except that the game challenger in Game 17 is split into the $\mathcal{F}_{\text{PAKE-1EA}}$ functionality and the simulator $\mathcal{S}$ in the ideal world (which, of course, does not make a difference in $\mathcal{Z}$'s view).

item to be simulated	case	Game 17 (Figure 5)	simulator (Figures 1 and 2)
P-to-P' message		line 2	the only step
P''s session key and P'-to-P message	$(s^*, T^*) = (s, T)$	line 4–7	step 1
	no $(\mathsf{pw}')^*$, or $(\mathsf{pw}')^* \neq \mathsf{pw}'$	line 10–13	step 2(b)(i)
	$(\mathsf{pw}')^* = \mathsf{pw}'$	line 14–25	step 2(b)(ii)
P's session key	$(s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$	line 27–30	step 1
	no pw^*, or more than one pw^*	line 33–34	step 2(a)
	no $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record	line 38–39	step 2(b)
	$K^* \neq K$	line 43–44	step 2(c)
	$\mathsf{pw}^* \neq \mathsf{pw}$	line 35–36	step 2(d)(i)
	$\mathsf{pw}^* = \mathsf{pw} \wedge K^* = K$	line 41–42	step 2(d)(ii)
$\mathcal{A}$'s $H_2(\star, T)$ queries		line 46–51	the only step

Table 1: Comparison between Game 17 and the ideal world. The "step" in simulator refers to the step in Figures 1 and 2 under the corresponding title

From Table 1 (and after going through some simple but tedious checks), we can see that Game 17 and the ideal world are exactly identical. (Various aborting conditions are omitted.) This completes the proof.

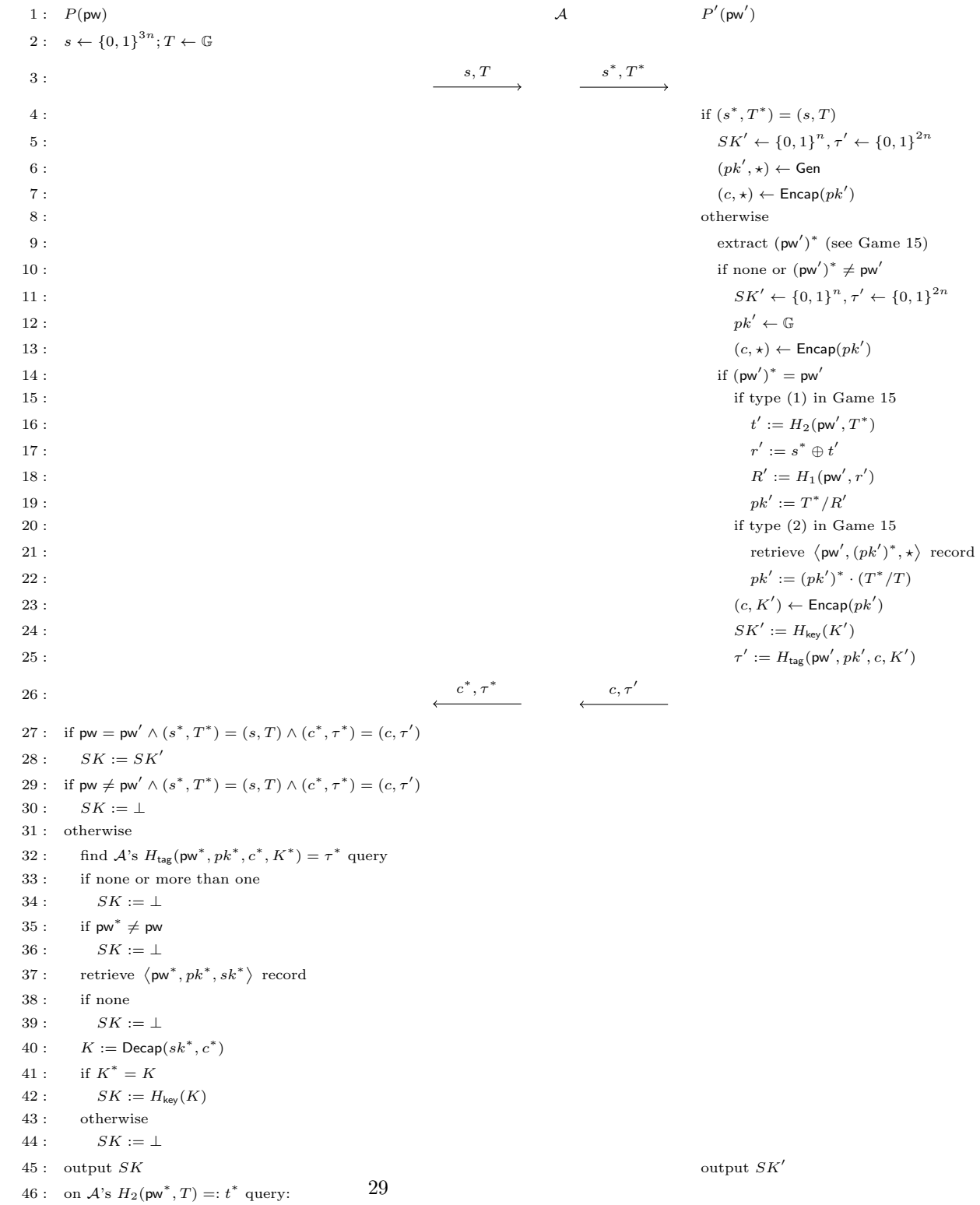

Figure 5: Game 17 (with various aborting conditions omitted)

4 Security Analysis of Other Versions

4.1 What Will Go Wrong If You Don't Hash Everything?

One might fear that every new variant of OEKE — with some inputs removed from H_{tag} — needs a new security analysis from scratch, causing this paper to be hundreds of pages long. Fortunately, most of the simulator and the game hops from the proof in Section 3 can be reused. This, in fact, should be expected: only the definition of the P'-to-P authenticator τ' changes in these variants, so the only parts of the proof that are affected are P's behavior — in particular, how to extract the password guess and when to output $\perp$ — on the P'-to-P message.

Concretely, Games 2, 11 and 13 in Section 3.2 need stronger KEM security properties, or at least a new analysis, to go through. We explained in *underlined* comments in each of them why the current arguments critically require pw, pk and/or c as part of the H_{tag} inputs, which we summarize in Table 2:

#	summary	what needs to be included in H_{tag}
2	extract K^* from τ^*	pw, pk, c
11	reject if $\mathcal{A}$ does not make an $H_2(\mathsf{pw}, T)$ query	pk
13	extract pw^* from τ^* and check against K^*	pw

Table 2: Summary of games in Section 3.2 that require pw, pk and/or c as part of H_{tag} inputs

4.2 The "Hash pw and pk" Version

We start from the version where τ is defined as $H_{\mathsf{tag}}(\mathsf{pw}, pk, K)$, i.e., c is removed from the hash. From Table 2, this is the easiest one to deal with: it only affects Game 2, and the simulator and all other games can be reused in the proof.

Looking closer, only Claim 1 in Game 2 needs a new proof: the argument after this claim also does not change if we remove c from H_{tag}. Below we state and prove the new claim.

Claim 2. *Suppose the WNM property holds for the KEM scheme. In the case where* $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$*, the probability that* P' *queries* $H_{\mathsf{tag}}(\mathsf{pw}, pk, K)$ *is negligible.*

P' makes a single H_{tag} query, on (pw', pk', K').

- If $\mathsf{pw} \neq \mathsf{pw}'$, then of course P''s query is not $H_{\mathsf{tag}}(\mathsf{pw}, pk, K)$.
- If $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T)$, then the argument in the proof of Claim 1 still goes through, so $\Pr[pk = pk'] \leq (q_1 + 1)/|\mathbb{G}|$.

The only remaining case is $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, which we now focus on. This implies that $pk = pk'$ by the definitions of pk and pk', and $c^* \neq c$ by the condition of this game. We now upper-bound $\Pr[K = K']$. Recall that K is defined as $\mathsf{Decap}(sk, c^*)$, so $\Pr[K = K'] = \Pr[\mathsf{Decap}(sk, c^*) = K']$. We upper-bound this probability via a reduction $\mathcal{R}_{\mathrm{WNM}}$ to the WNM property of KEM:

Reduction $\mathcal{R}_{\mathrm{WNM}}$:

On input (pk, c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T), except that $\mathcal{R}_{\mathrm{WNM}}$ uses its input pk rather than $(pk, \star) \leftarrow \mathsf{Gen}$. Send (s, T) to $\mathcal{A}$.
2. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise generate $(c, K') \leftarrow \mathsf{Encap}(pk)$, and compute $SK := H_{\mathsf{key}}(K')$ and $\tau := H_{\mathsf{tag}}(\mathsf{pw}, pk, c, K')$. Output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$.
3. On (c^*, τ^*) from $\mathcal{A}$, if $c^* = c$, abort. Otherwise output c^*.

The definition of $\mathcal{R}_{\mathrm{WNM}}$ ensures that $\mathsf{Decap}(sk, c) = K'$. If $\mathcal{A}$ sends c^* such that $\mathsf{Decap}(sk, c^*) = K'$, then $\mathcal{R}_{\mathrm{WNM}}$ wins the WNM game. Thus,

$$\Pr[\mathsf{Decap}(sk, c^*) = K'] \leq \mathbf{Adv}^{\mathrm{WNM}}_{\mathcal{R}_{\mathrm{WNM}}}.$$

If $\mathsf{Decap}(sk, c) \neq K'$, i.e., $K \neq K'$, then of course P''s query is not $H_{\mathsf{tag}}(\mathsf{pw}, pk, K)$. This yields the claim.

4.3 The “Hash pw” Version

We now turn to the variant where pk is further removed from H_{tag} inputs, i.e., τ is defined as $H_{\mathsf{tag}}(\mathsf{pw}, K)$. Table 2 suggests that Games 2 and 11 in Section 3.2 need to (essentially) change. In fact the simulator also needs to change quite a bit, which we will start from.

Changes to the simulator. The only changes to the simulator $\mathcal{S}$ appear when $\mathcal{A}$ sends (c^*, τ^*) to P and $\mathcal{S}$ needs to extract a password guess pw^* from it. In the “hash everything” version (Section 3.1), τ^* is computed as $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, c^*, K^*)$, so $\mathcal{S}$ can extract pw^* from $\mathcal{A}$'s H_{tag} queries and *check its consistency with* pk^*, c^*, K^* (see the last paragraph of Section 3.1.1). Here, $\tau^* = H_{\mathsf{tag}}(\mathsf{pw}^*, K^*)$, but the inclusion of pw^* still allows $\mathcal{S}$ to find the corresponding $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record and check for consistency.

However, a more sinister issue emerges when we remove pk^* from H_{tag}: In the case where the two parties' passwords match, $\mathcal{A}$ can modify c and force the two parties' session keys to be the same — without making *any* H_{tag} query! The easiest way to see this attack is to work out a concrete version on OEKE based on the Diffie-Hellman KEM, which we show in Figure 6.

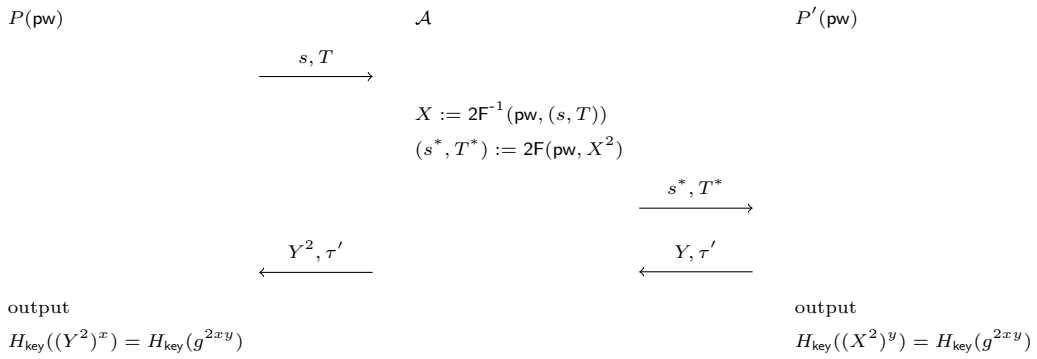

Figure 6: New attack on the "hash pw" version

In this attack, the two honest parties' passwords pw are the same. Assuming $\mathcal{A}$ knows it, it can decrypt P's message to recover $X = g^x$, and then re-encrypt X^2 under the same pw; this causes P' to output $SK' = H_{\mathsf{key}}(g^{2xy})$ and send $(Y = g^y, \tau' = H_{\mathsf{tag}}(\mathsf{pw}, g^{2xy}))$. Then $\mathcal{A}$ sends (Y^2, τ') to P, causing P's check to pass and P will also output $H_{\mathsf{key}}(g^{2xy})$. (This is not an issue if pk is included in H_{tag}, since then P would compute $\tau = H_{\mathsf{tag}}(\dots, Y^2, \dots) \neq H_{\mathsf{tag}}(\dots, Y, \dots) = \tau'$ and its check would fail.)

In fact, it is a stretch to call this an attack, since the only thing $\mathcal{A}$ does is to modify some protocol messages and cause P and P' to output the same session key — which $\mathcal{A}$ *cannot* compute. Still, this scenario is not covered by the previous simulator. It should be simulated as follows: When $\mathcal{A}$ decrypts (s, T) under pw, $\mathcal{S}$ records $\langle \mathsf{pw}, pk, sk \rangle$ (although there might be a large number of such records, and $\mathcal{S}$ does not know which is the "right" one at this point). Then when $\mathcal{A}$ sends (s^*, T^*) to P', $\mathcal{S}$ extracts pw, which allows $\mathcal{S}$ to retrieve the "right" record $\langle \mathsf{pw}, pk, sk \rangle$; $\mathcal{S}$ also compute K' while simulating the P' side. Later, when $\mathcal{A}$ sends (c^*, τ'), $\mathcal{S}$ can tell if $\mathcal{A}$ is performing the aforementioned attack by checking if $\mathsf{Decap}(sk, c^*) = K'$.

We show the new simulator in Figure 7. (Only the steps after receiving (c^*, τ^*) are included.)

P's session key

On (c^*, τ^*) from $\mathcal{F}$, simulate P's output as follows:

1. If $(s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$. // $\mathcal{A}$ is passive
2. If $(s^*, T^*) \neq (s, T) \wedge \tau^* = \tau'$, and there has been a $(\mathsf{TestPwd}, sid, P', (\mathsf{pw}')^*)$ command to $\mathcal{F}$ resulting in "correct guess" (i.e., $\mathcal{S}$ previously entered step 2(b)(ii) in Figure 2), there must be a K' defined at that time.
   (a) Retrieve record $\langle (\mathsf{pw}')^*, \star, sk \rangle$. If there is no such record, go to step 3.
   (b) Compute $K := \mathsf{Decap}(sk, c^*)$. If $K \neq K'$, go to step 3.
   (c) Send $(\mathsf{TestPwd}, sid, P', (\mathsf{pw}^*)')$ to $\mathcal{F}$.
      i. If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
      ii. If $\mathcal{F}$ replies with "correct guess", compute $SK := H_{\mathsf{key}}(K)$ and send $(\mathsf{NewKey}, sid, P', SK)$ to $\mathcal{F}$.
3. Otherwise find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}^*, K^*)$ query resulting in τ^*.
   (a) If there is no such query or more than one such query, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
   (b) If there is exactly one such query, retrieve record $\langle \mathsf{pw}^*, \star, sk^* \rangle$. If there is no such record, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
   (c) Compute $K := \mathsf{Decap}(sk^*, c^*)$. If $K^* \neq K$, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
   (d) Send $(\mathsf{TestPwd}, sid, P', \mathsf{pw}^*)$ to $\mathcal{F}$.
      i. If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
      ii. If $\mathcal{F}$ replies with "correct guess", compute $SK := H_{\mathsf{key}}(K)$ and send $(\mathsf{NewKey}, sid, P', SK)$ to $\mathcal{F}$.

Figure 7: UC simulator $\mathcal{S}$ for the "hash pk" variant of OEKE. Only the paragraph that needs significant change (shown in blue) is included

4.4 The "Hash pk" Version

We now consider the variant where pw, but not pk, is further removed from H_{tag} inputs, i.e., τ is defined as $H_{\mathsf{tag}}(pk, K)$. Table 2 suggests that Games 2 and 13 in Section 3.2 need to (essentially) change; however, we will (again) start from the changes to the simulator.

Changes to the simulator. Similar to Section 4.3, once we remove pk from H_{tag}, $\mathcal{A}$ can force the two parties' session keys to be the same without performing a "real" attack. However, this time the two parties' passwords are different (and

$\mathcal{A}$ needs to know both of them), and $\mathcal{A}$ should forward (c, τ') from P' without modifications. We show this attack in Figure 8.

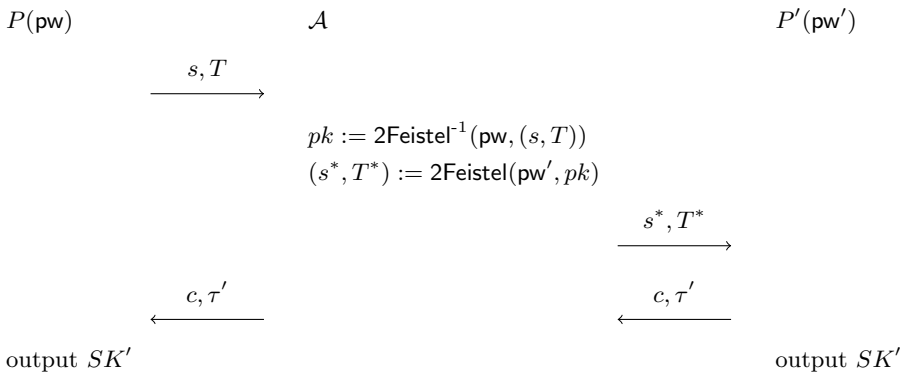

Figure 8: New attack in the "hash pk" version

In this attack, the two honest parties' passwords pw and pw' are different. Assuming $\mathcal{A}$ knows both, it can decrypt P's message to recover pk, and then re-encrypt pk under pw'; this ensures that P' will recover the same pk as on the P side, and use it to compute c, K', τ' (and eventually SK'). But then if $\mathcal{A}$ forwards (c, τ') without any modification, P will decapsulate c to the same K' (because it uses the same pk), and its output key will be the same as SK' on the P' side. (This is not an issue if pw is included in H_{tag}, since forwarding $\tau' = H_{\mathsf{tag}}(\mathsf{pw}', \dots) \neq H_{\mathsf{tag}}(\mathsf{pw}, \dots)$ to P would make P's check fail.)

This is (again) a new situation not covered by the previous simulator. It should be simulated as follows: When $\mathcal{A}$ decrypts (s, T) under pw, $\mathcal{S}$ records $\langle \mathsf{pw}, pk, sk \rangle$ (although there might be a large number of such records, and $\mathcal{S}$ does not know which is the "right" one at this point). Then when $\mathcal{A}$ sends (s^*, T^*) to P', $\mathcal{S}$ extracts pw', *which also gives a* pk'. Later, when $\mathcal{A}$ forwards (c, τ'), $\mathcal{S}$ knows that $\mathcal{A}$ is performing the aforementioned attack, and checks which $\langle \mathsf{pw}, pk, sk \rangle$ record matches pk'. In this way $\mathcal{S}$ can extract the password guess pw on the P side.

We show the new simulator in Figure 9. (Only the steps after receiving (c^*, τ^*) are included.)

P's session key

On (c^*, τ^*) from $\mathcal{F}$, simulate P's output as follows:

1. If $(s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$. // $\mathcal{A}$ is passive
2. If $(s^*, T^*) \neq (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, and there has been a $(\mathsf{TestPwd}, sid, P', (\mathsf{pw}')^*)$ command to $\mathcal{F}$ resulting in "correct guess" (i.e., $\mathcal{S}$ previously entered step 2(b)(ii) in Figure 2), there must be a pk' and an SK' defined at that time. If there is more than one record in the form of $\langle \mathsf{pw}^*, pk', \star \rangle$, abort. If there is a unique such record, send $(\mathsf{TestPwd}, sid, P, \mathsf{pw}^*)$ to $\mathcal{F}$.
   (i) If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
   (ii) If $\mathcal{F}$ replies with "correct guess", send $(\mathsf{NewKey}, sid, P, SK')$ to $\mathcal{F}$.
3. Otherwise find $\mathcal{A}$'s $H_{\mathsf{tag}}(pk^*, K^*)$ query resulting in τ^*.
   (a) If there is no such query or more than one such query, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
   (b) If there is exactly one such query, retrieve record $\langle \mathsf{pw}^*, \star, sk^* \rangle$. If there is more than one such record, abort. If there is no such record, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
   (c) Compute $K := \mathsf{Decap}(sk^*, c^*)$. If $K^* \neq K$, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
   (d) Send $(\mathsf{TestPwd}, sid, P', \mathsf{pw}^*)$ to $\mathcal{F}$.
      i. If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
      ii. If $\mathcal{F}$ replies with "correct guess", compute $SK := H_{\mathsf{key}}(K)$ and send $(\mathsf{NewKey}, sid, P', SK)$ to $\mathcal{F}$.

Figure 9: UC simulator $\mathcal{S}$ for the "hash pk" variant of OEKE. Only the paragraph that needs significant change (shown in blue) is included

References

[ABJ25] Afonso Arriaga, Manuel Barbosa, and Stanislaw Jarecki. NoIC: PAKE from KEM without ideal ciphers. Cryptology ePrint Archive, Report 2025/231, 2025.

[BS23] Dan Boneh and Victor Shoup. A graduate course in applied cryptography (version 0.6), 2023. https://toc.cryptobook.us/book.pdf.

[JRX25] Jake Januzelli, Lawrence Roy, and Jiayu Xu. Under what conditions is encrypted key exchange actually secure? In *EUROCRYPT 2025*, pages 451–481, 2025.

[MRR20] Ian McQuoid, Mike Rosulek, and Lawrence Roy. Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In *CCS 2020*, pages 425–442, 2020.

A Comparison Between Our Simulator and the One in [ABJ25]

Our simulator in Section 3.1.2 is, by and large, the same as the one in [ABJ25, Fig.5 and 6], although we completely rewrite it to improve readability. We first summarize some of the notational differences in Table 3.

	our notation	notation in [ABJ25]	comment
honest parties	P, P'	A, B	easy to confuse with adversary $\mathcal{A}$
simulated P-to-P' message	(s, T)	$(\overline{s}, \overline{T})$	
adversarial P-to-P' message	(s^*, T^*)	(s, T)	
$\mathcal{A}$'s guess of P''s password	$(\mathsf{pw}')^*$	pw^*	
KEM public key corresponding to correct password guess on P'	pk'	pk^*	
simulated P'-to-P message	(c, τ')	(c, tag)	
adversarial P'-to-P message	(c^*, τ^*)	(c, tag)	same (c, tag) for different things
$\mathcal{A}$'s guess of P's password	pw^*	pw^*	same pw^* for different things

Table 3: Comparison of notations between our simulator and the one in [ABJ25]

The simulator in [ABJ25] has more bookkeeping and aborts more frequently. In particular, we completely remove their record forwS, and when detecting whether an adversarial P-to-P' message contains more than one password guess (causing the simulator to abort), we only check the relevant H_2 queries, whereas they check *all* H_2 queries — including those that are completely unrelated to the attack in the P-to-P' direction. For a more detailed explanation, see the comment after Game 15 in Section 3.2.

The only (more or less) essential difference lies in the red text in Figure 1: in the case where the adversary makes an incorrect password guess on P', we compute c from a uniformly random KEM public key pk', whereas they compute c from an honestly generated pk'. While both methods work, our method simplifies the reduction; this is further explained in the comment after Game 18 in Section 3.2, item (2).